



北京航空航天大学
BEIHANG UNIVERSITY



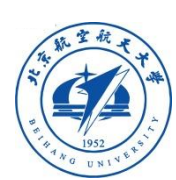
微机原理与接口技术

第六章 (2)

林新

Lx@buaa.edu.cn

北京航空航天大学 自动化学院



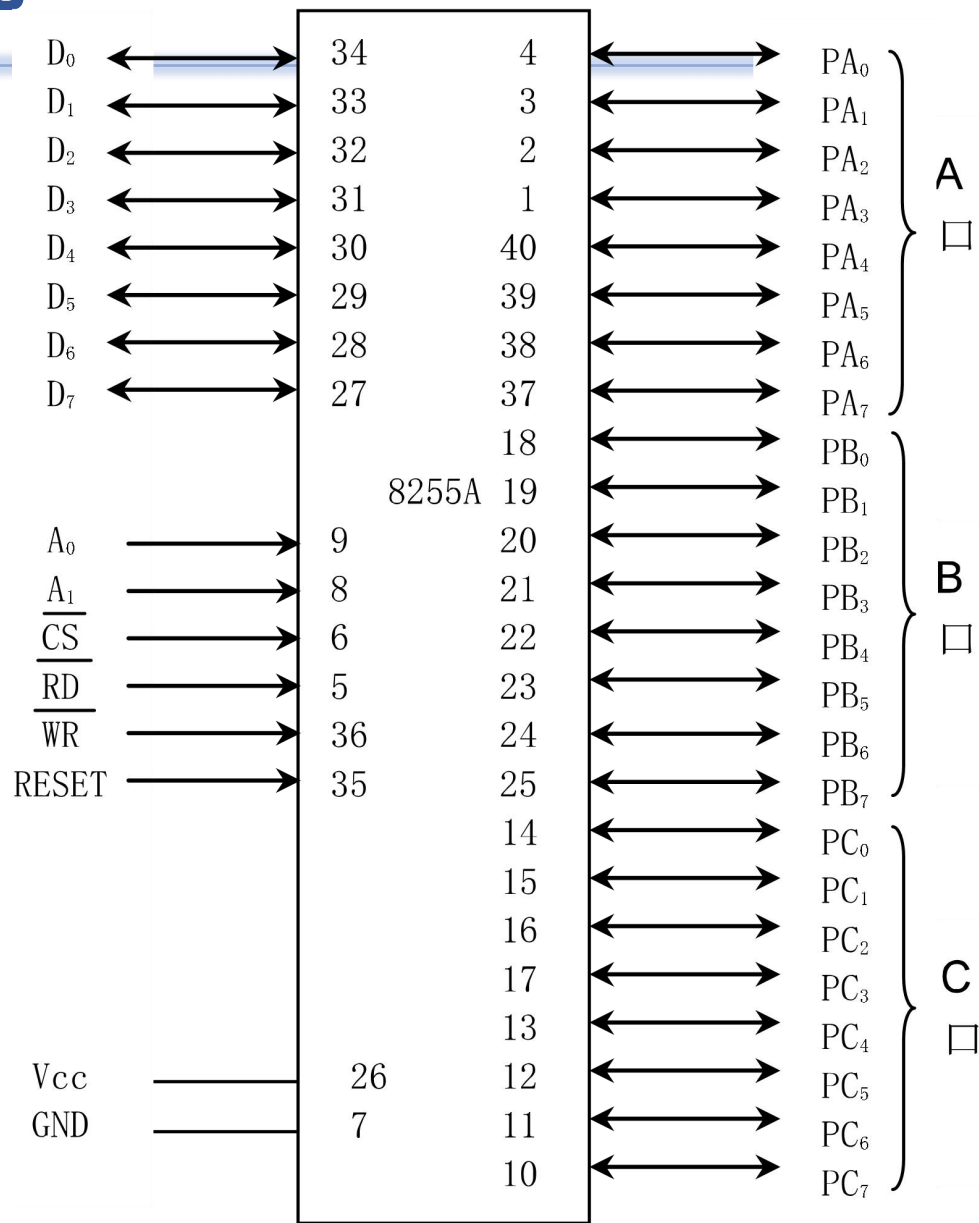
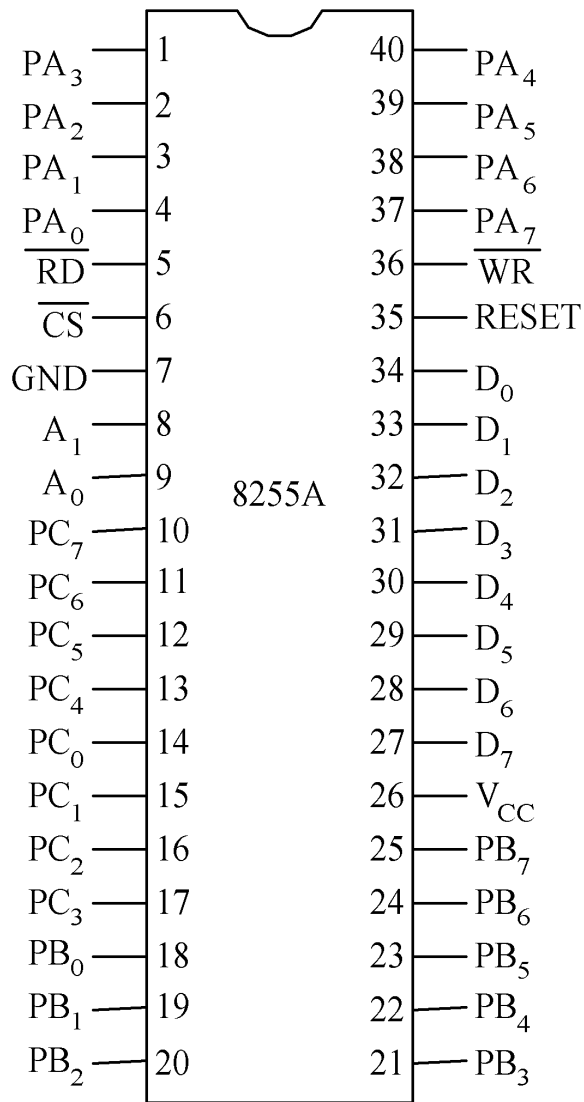
6.2 8255A的工作原理

并行接口芯片8255A:

- 掌握8255A的工作方式0
- 掌握利用8255A进行键盘识别、显示的方法



6.2.1 结构与功能

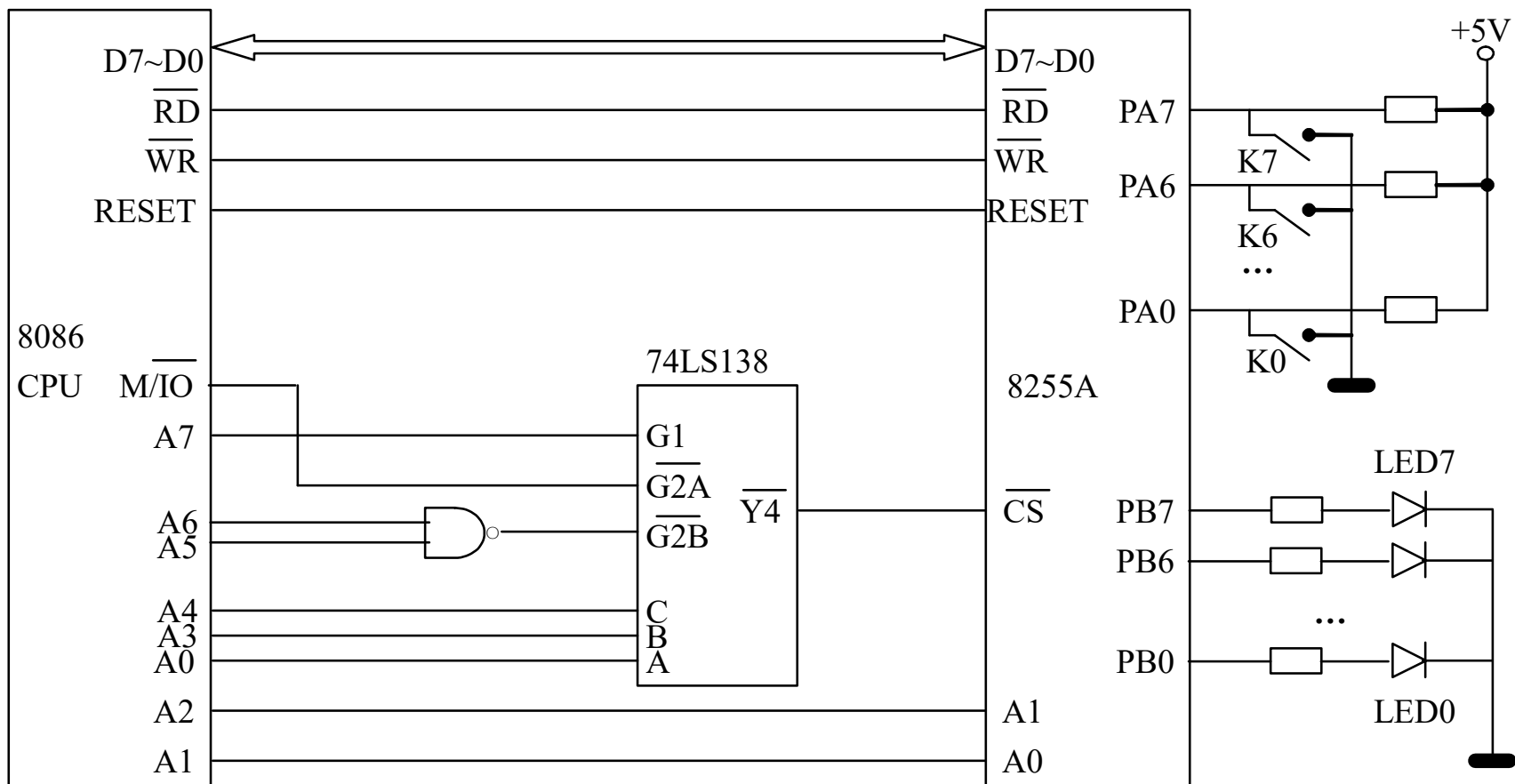


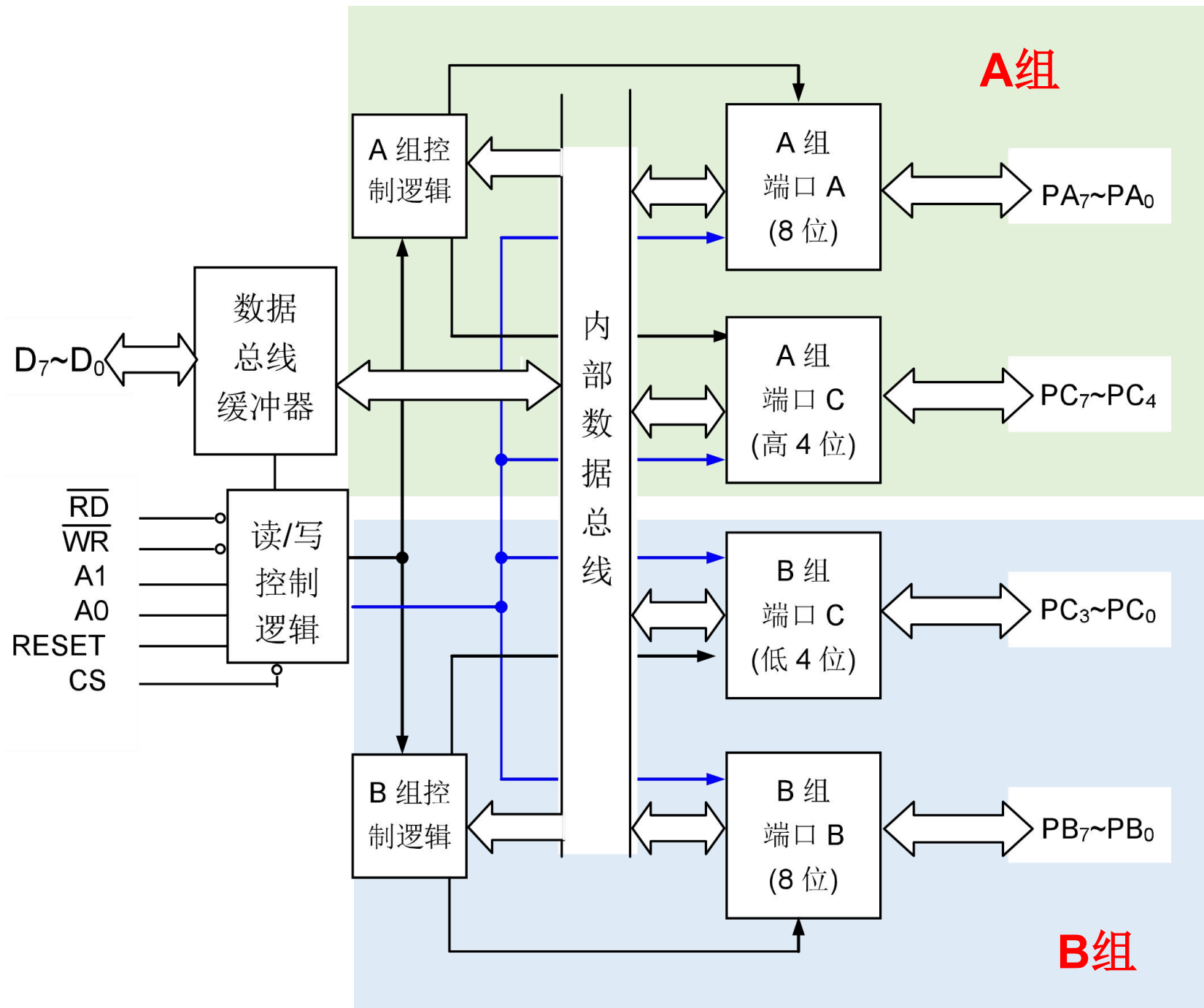
与CPU连接的引脚

与外设连接的引脚



8255A与CPU连接示例







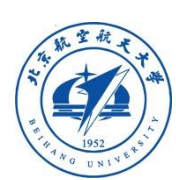
1. 并行I/O端口A、B、C

- 8255A内部有3个8位输入输出端口：A、B、C
- 端口C可以作为8位端口统一使用，也可以分为高4位和低4位两部分，分别独立使用
- 与三个内部端口对应引脚分别为：PA7~PA0, PB7~PB0, PC7~PC0



2. A组和B组控制逻辑

- 8255A内部有两组控制PA、PB、PC端口的逻辑电路：A组和B组控制逻辑
 - A组逻辑用于控制PA端口，PC端口高4位
 - B组逻辑用于控制PB端口，PC端口低4位
- CPU向控制逻辑电路中的控制寄存器写入工作方式字，可以控制PA、PB、PC端口的工作方式



3.数据总线缓冲器

- 双向三态缓冲器，用作8255A与系统总线之间的接口

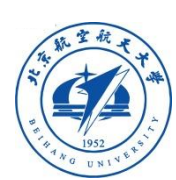


4.读/写控制逻辑

- $\overline{CS}$: 片选信号, 低电平有效
- $\overline{RD}$: 读信号, 低电平有效
- $\overline{WR}$: 写信号, 低电平有效
- A1A0: 端口选择信号
 - A1A0=00 PA, 01 PB, 10 PC, 11 控制端口
- RESET: 复位信号, 高电平有效, 将8255A内部寄存器清0.

A1	A0	$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	操作
0	0	0	1	0	端口 A→数据总线
0	1	0	1	0	端口 B→数据总线
1	0	0	1	0	端口 C→数据总线
					} 读操作
0	0	1	0	0	数据总线→端口 A
0	1	1	0	0	数据总线→端口 B
1	0	1	0	0	数据总线→端口 C
1	1	1	0	0	数据总线→控制字寄存器
					} 写操作
×	×	×	×	1	$D_7 \sim D_0$ 数据总线呈高阻状态
1	1	0	1	0	非法状态
×	×	1	1	0	$D_7 \sim D_0$ 数据总线呈高阻状态

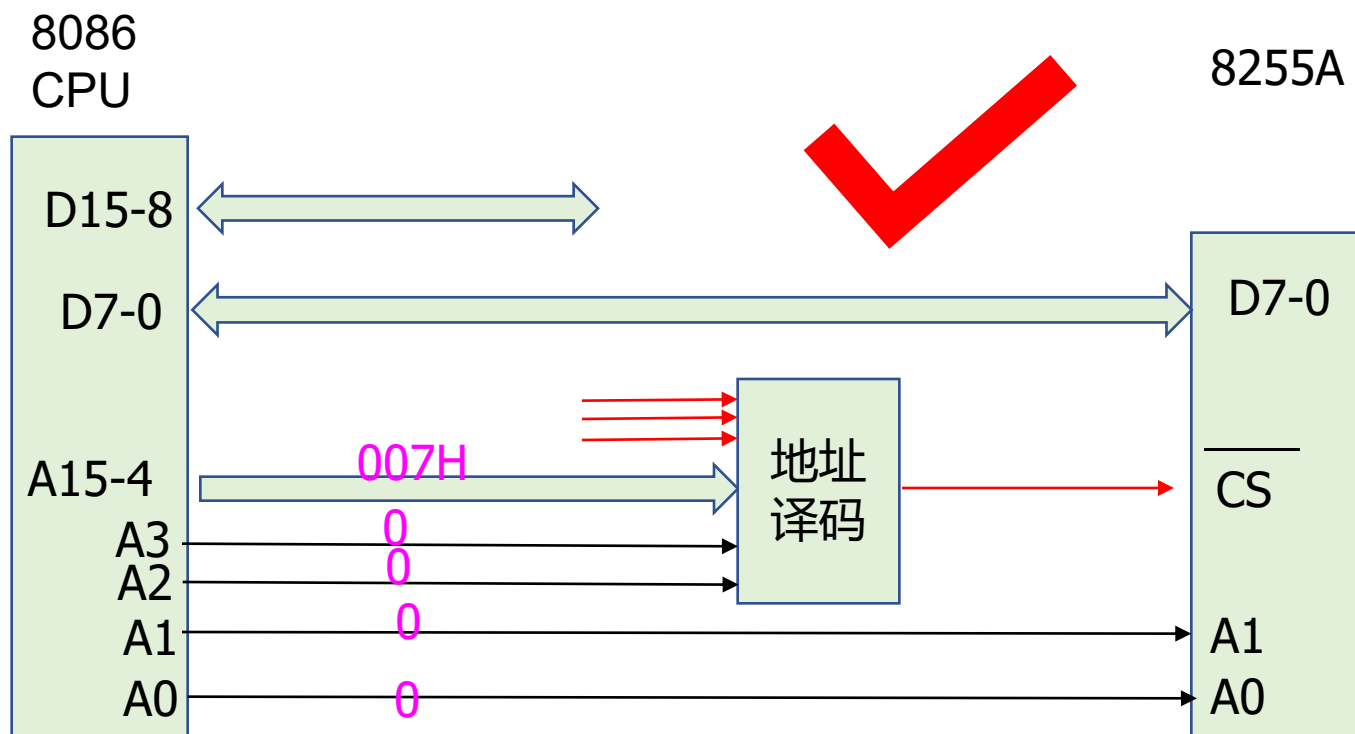
- A1 , A0与 $\overline{CS}$, $\overline{RD}$, $\overline{WR}$ 信号一起, 用来确定8255A的操作状态



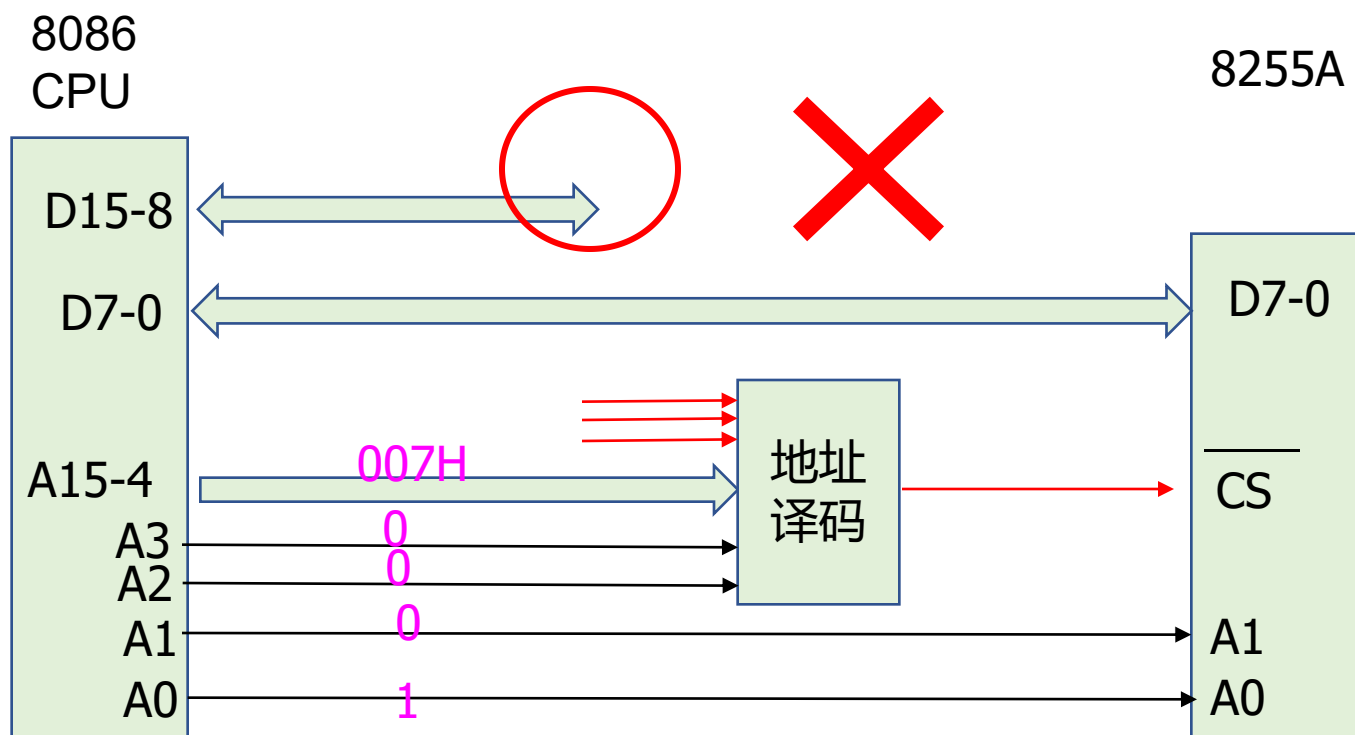
- 当8255A用在8位数据总线的微处理器系统(如8088)时，端口选择信号输入端A1A0分别与地址总线的A1A0相连即可
- 在16位数据总线的系统（8086）中，通常将**地址总线的A2,A1**连到**8255A的A1,A0**端，以便8255A的数据引脚连接在CPU数据总线的低8位上，用偶地址来寻址8255A

系统连线如下图所示。设8255A端口A、B、C、以及控制端口的地址分别为70H，71H，72H，73H；AL=12H。

则执行下面指令时：OUT 70H, AL



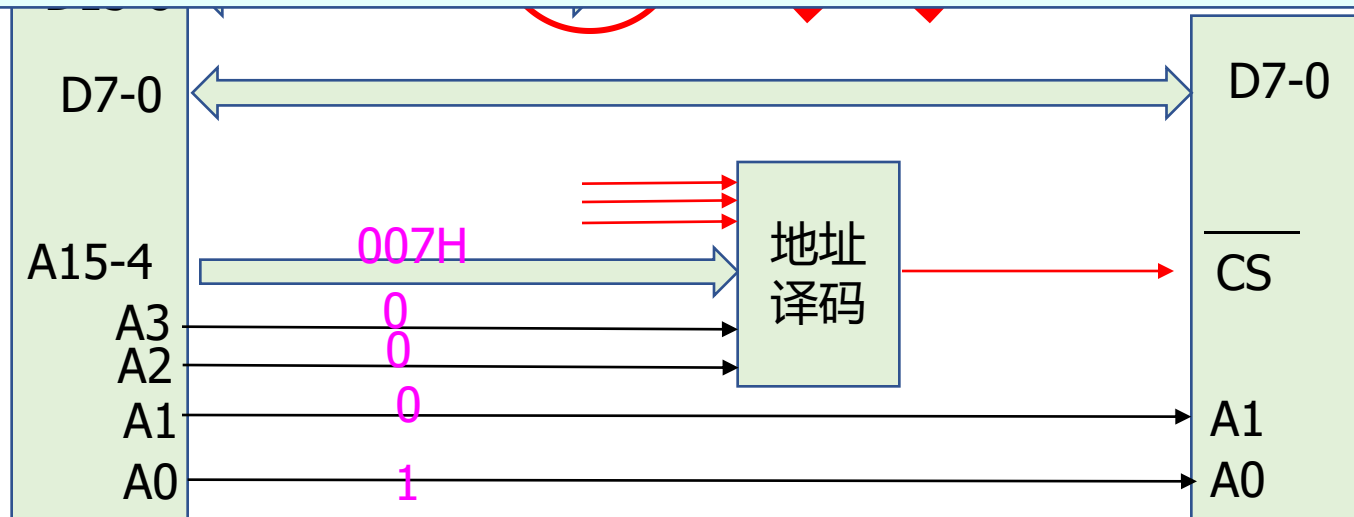
则执行下面指令时：OUT 71H, AL



结论1:

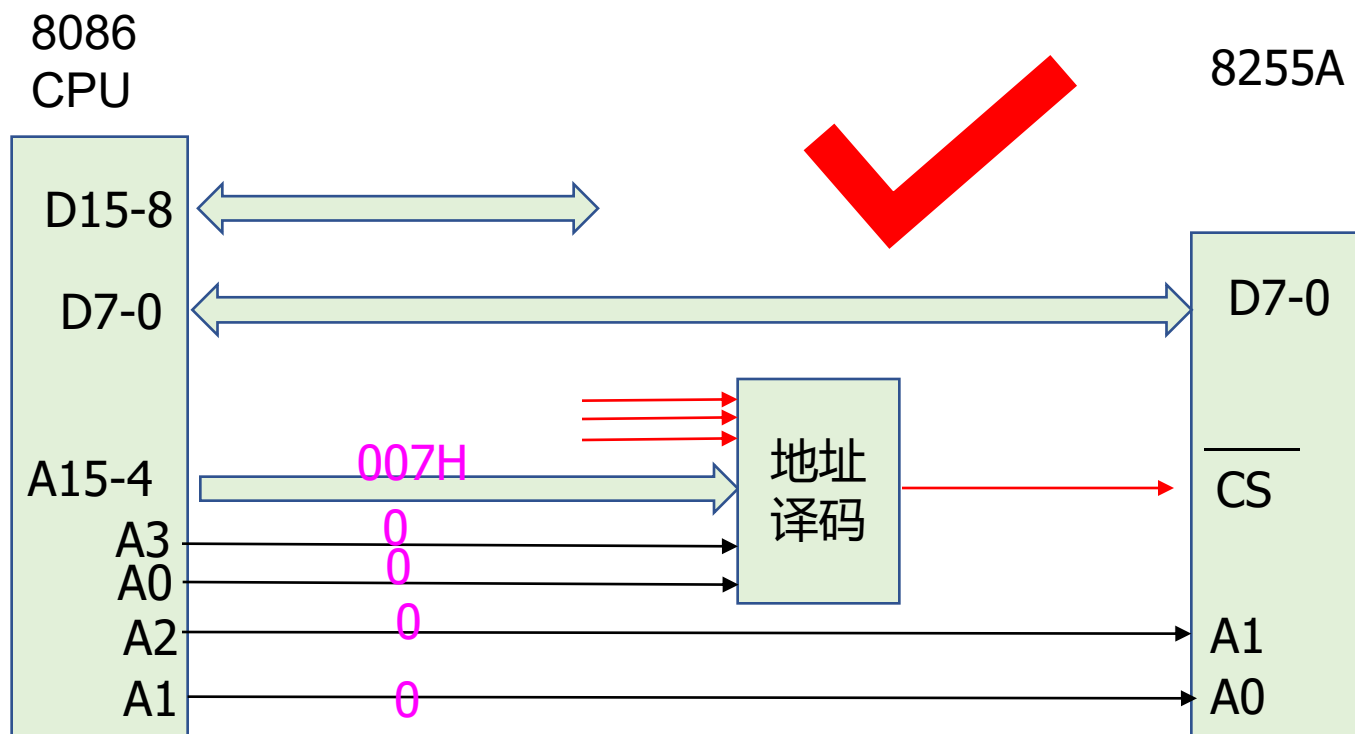
由于**8255A数据引脚** (D7~D0) 连接到8086系统时, 选择了**数据总线低8位**,

因此若**8255A的A1A0**直接连接8086系统**地址总线A1A0**, 使用连续端口地址, 会导致奇地址端口的**数据读写出错**。



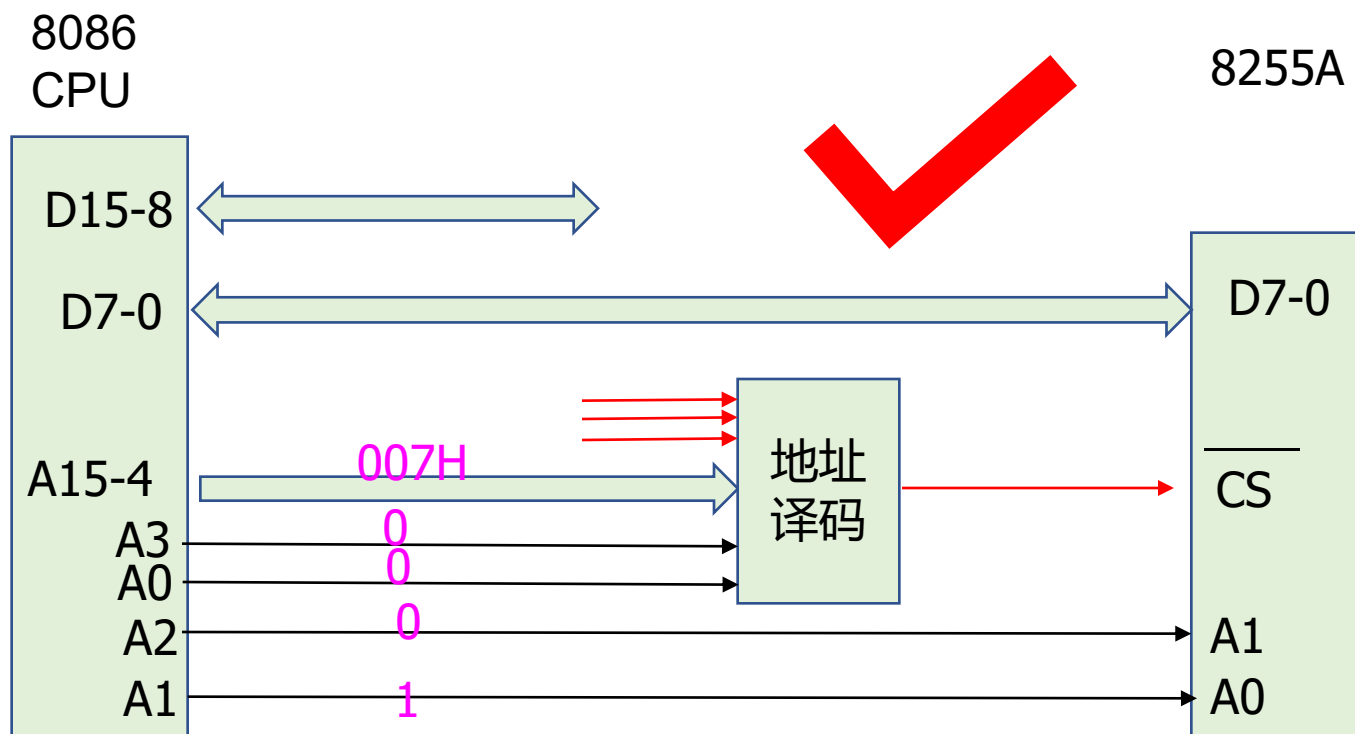
系统连线如下图所示。设8255A端口A、B、C、以及控制端口的地址分别为**70H**，**72H**，**74H**，**76H**；AL=12H。

则执行下面指令时：OUT **70H**, AL



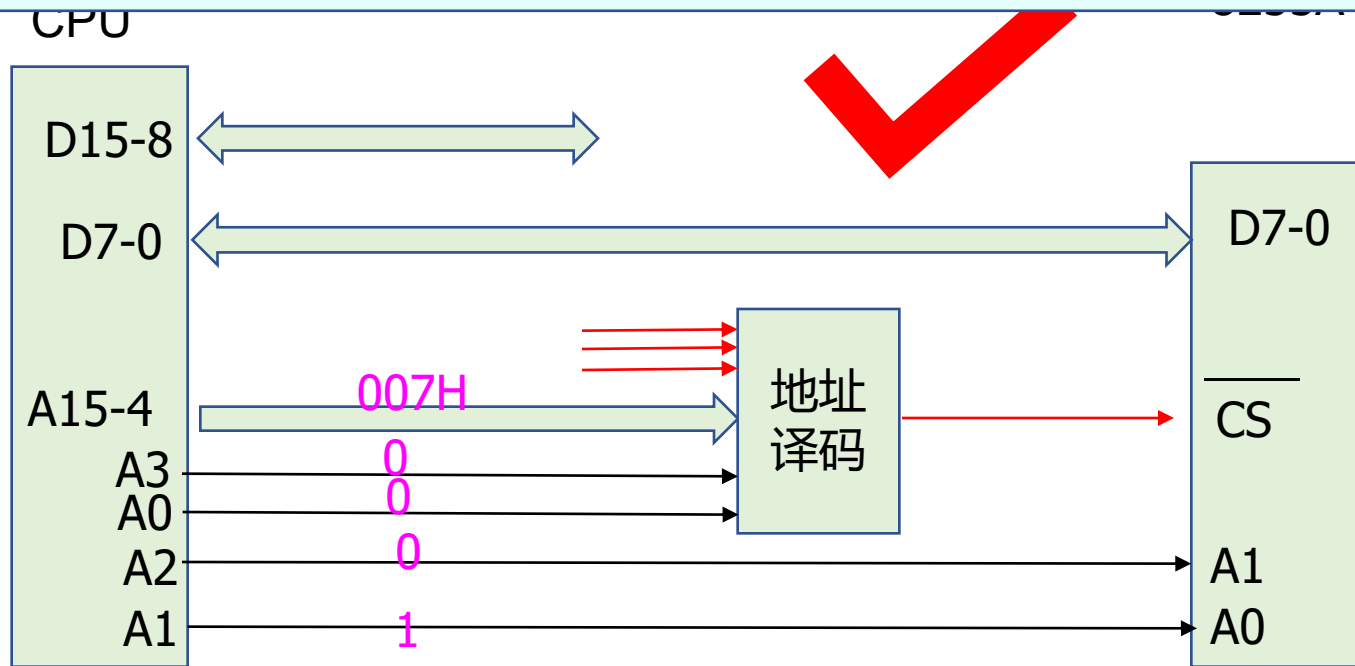
系统连线如下图所示。设8255A端口A、B、C、以及控制端口的地址分别为**70H**，**72H**，**74H**，**76H**；AL=34H。

则执行下面指令时：OUT **72H**, AL

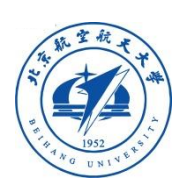


结论2:

设计端口译码电路，使得端口地址**都是偶地址**
(使用数据总线低8位时传送数据)，端口即可正
常读写的数据

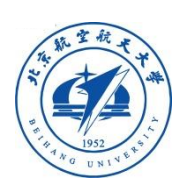


推论：如果使用**数据总线高8位**传送数据，则需要把
端口地址设计为奇地址



结论：

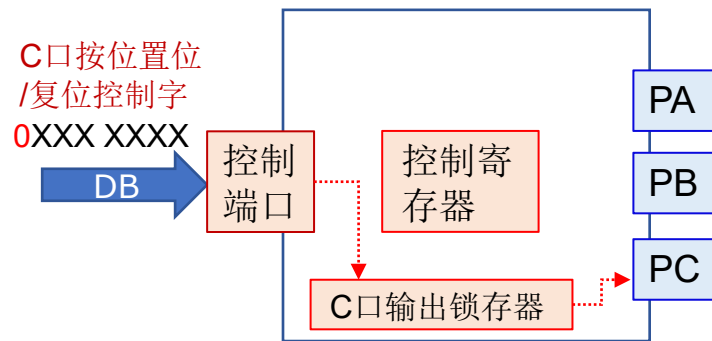
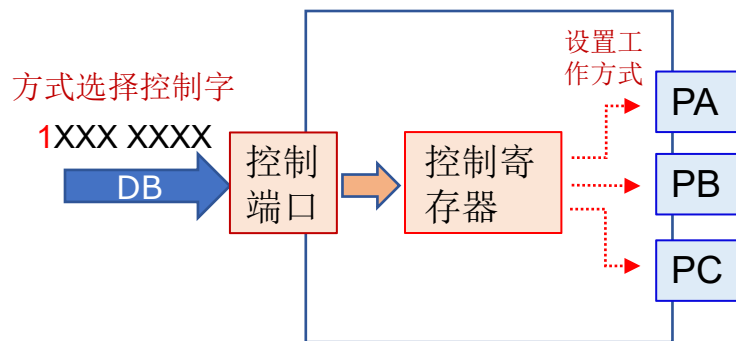
- **8255A数据引脚** (D7~D0) 通过**数据总线低8位**连接到8086系统时，若**8255A的A1A0**直接连接8086系统**地址总线A1A0**，使用连续端口地址，会导致奇地址的**数据读写出错**。为此，需要设计端口译码电路，使得端口地址都是偶地址。
- 如果使用数据总线高8位传输数据，则需要将端口设置为奇地址
- 其它**接口电路芯片**情况类似。



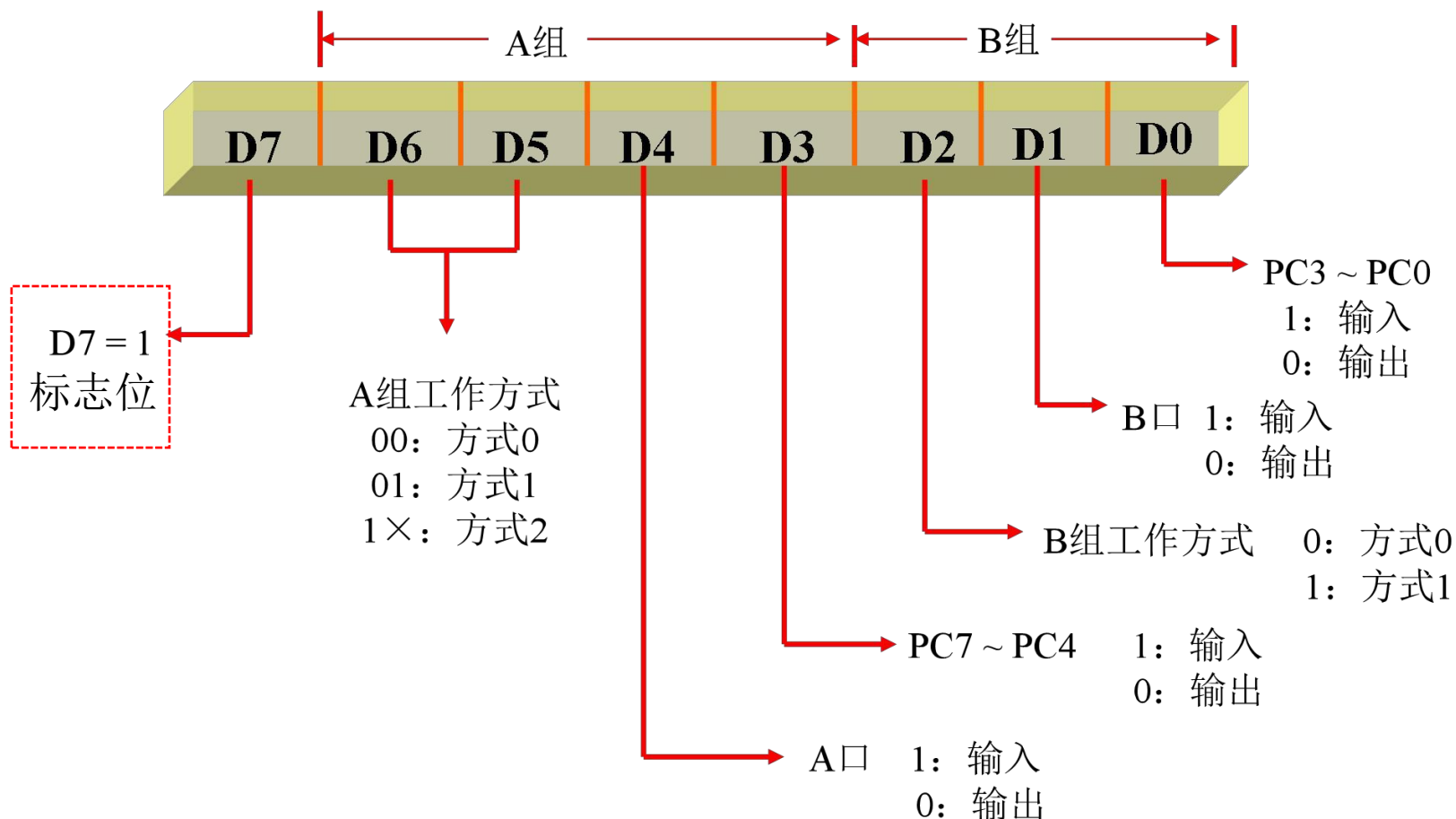
6.2.2 8255A控制字

写入8255A控制端口的**控制字**长度为1个字节，有两种：

- **方式选择控制字**：最高位 $D_7=1$ ，用来设定PA、PB、PC端口的工作方式、数据传送方向。
- **C口按位置位/复位控制字**：最高位 $D_7=0$ ，用于设置PC端口**指定引脚**的输出1(置位)或0(复位)。



1、8255的方式字（写入 A1、A0 = 11）

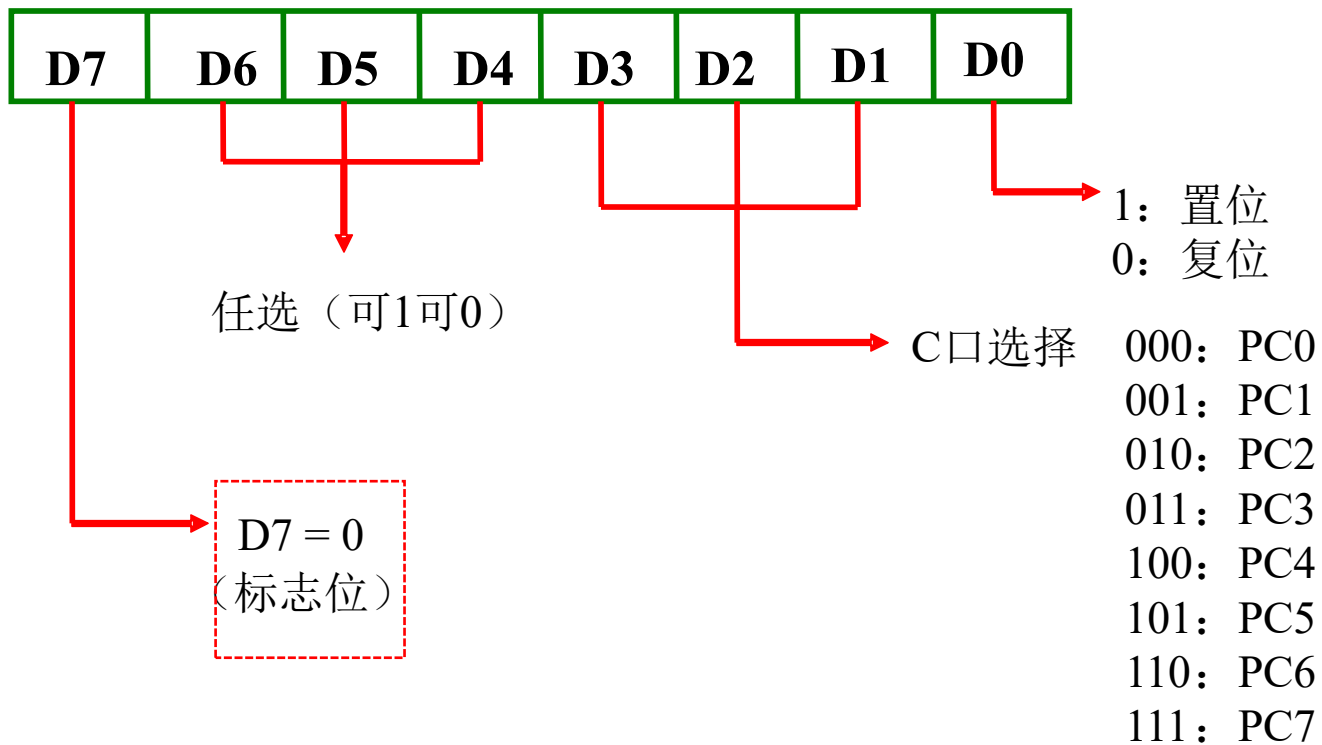


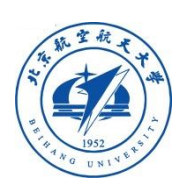
方式0: 基本 I/O 方式（查询时，**任选**C口做联络信号）

方式1: 选通I/O 方式（可查询或中断，**固定**C口做联络信号）

方式2: 双向I/O方式（只用于A口，PC3 ~ PC7做联络信号）

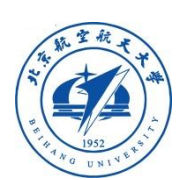
2、C口的置位/复位字（写入 A1A0 = 11）





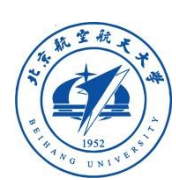
6.2.4 8255A工作方式

- (1)工作方式0：基本输入输出方式
- (2)工作方式1：选通输入输入输出方式
- (3)工作方式2：双向传送方式

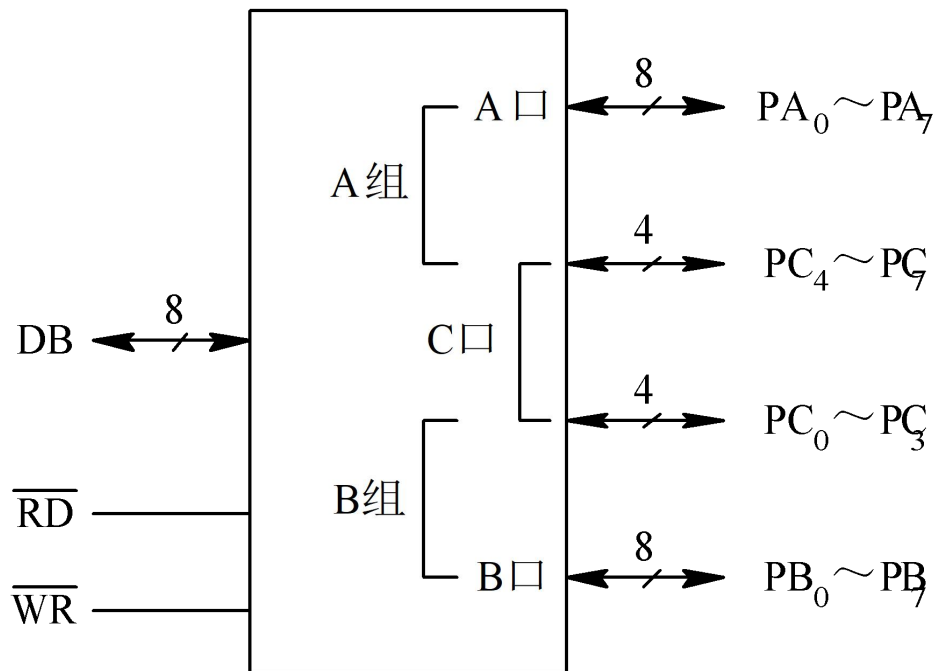
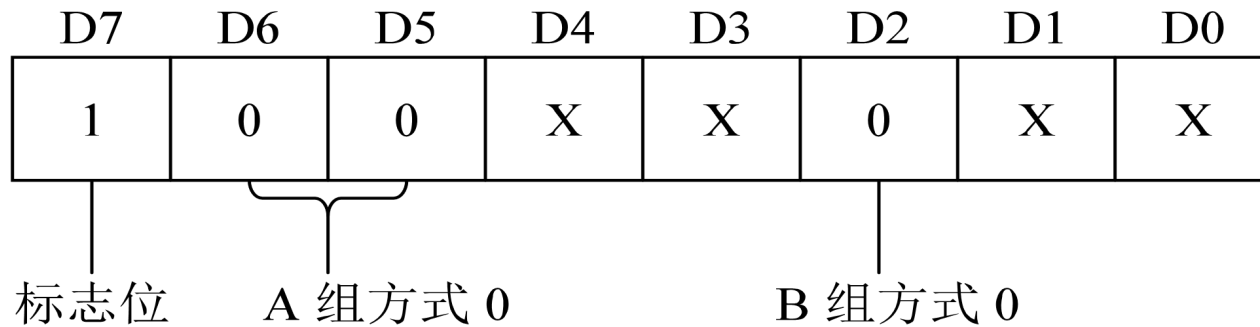


方式0：基本输入输出方式

- 适用于不需要应答信号的简单输入输出场合。
- 三个端口都可以由程序规定为输入或输出方式，但不能既作为输入又作为输出，也没有提供固定的联络信号。
- C口分为两个4位：高4位和低4位，可以分别设置为输入或输出方式。
- 各端口在输出方式下有锁存功能，输入不锁存



■ 各端口都工作在方式0的方式选择字格式





例：假设8255A的控制字寄存器的端口地址为66H，若要求A组和B组工作于方式0，A口、C口的上半部分（高4位）作输入，B口和C口的下半部分（低4位）为输出。

1) 编程设置8255A

2) 若从A口读入的数据为31H，则向B口输出FFH，否则输出0；
若从PC7读入的值为1，则向PC3输出1，否则输出0

(1) 确定端口地址。**控制端口**,PC,PB,PA的端口地址为从大到小连续的偶地址：**66H→64H→62H→60H**

(2) 确定工作方式字

标志位	A组工作方式		PA 1输入 0输出	PC _{7~4} 1输入 0输出	B组工作方式	PB 1输入 0输出	PC _{3~0} 1输入 0输出
D7=1	D6	D5	D4	D3	D2	D1	D0
1	0	0	1	1	0	0	0

98H



例：假设8255A的控制字寄存器的端口地址为66H，若要求A组和B组工作于方式0，A口、C口的上半部分（高4位）作输入，B口和C口的下半部分（低4位）为输出。

1) 编程设置8255A

2) 若从A口读入的数据为31H，则向B口输出FFH，否则输出0；
若从PC7读入的值为1，则向PC3输出1，否则输出0

(3) 编程，设置8255A工作方式

```
MOV AL, 98H
```

```
OUT 66H, AL
```

(4) 编程，完成输入输出任务

```
IN AL, 60H
```

```
CMP AL, 31H
```

```
JE NXT1
```

```
MOV AL, 0
```

```
JMP NXT2
```

```
NXT1: MOV AL, 0FFH
```

```
NXT2: OUT 62H, AL
```

```
IN AL, 64H
```

```
TEST AL, 80H
```

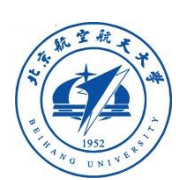
```
JNZ NXT3
```

```
MOV AL, 0000 0110B
```

```
JMP NXT4
```

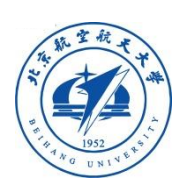
```
NXT3: MOV AL, 0000 0111B
```

```
NXT4: OUT 66H, AL
```



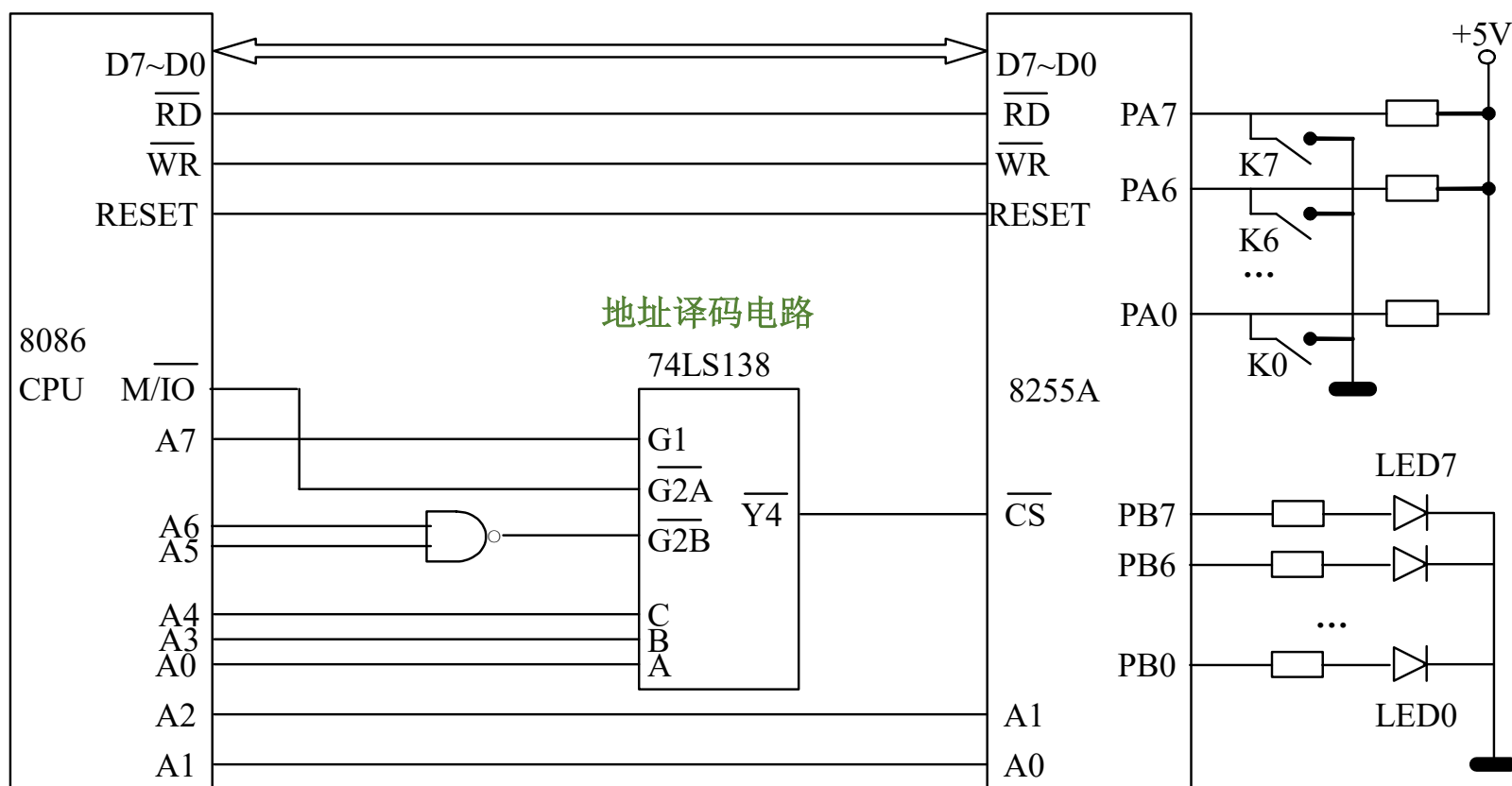
6.3 8255A应用举例

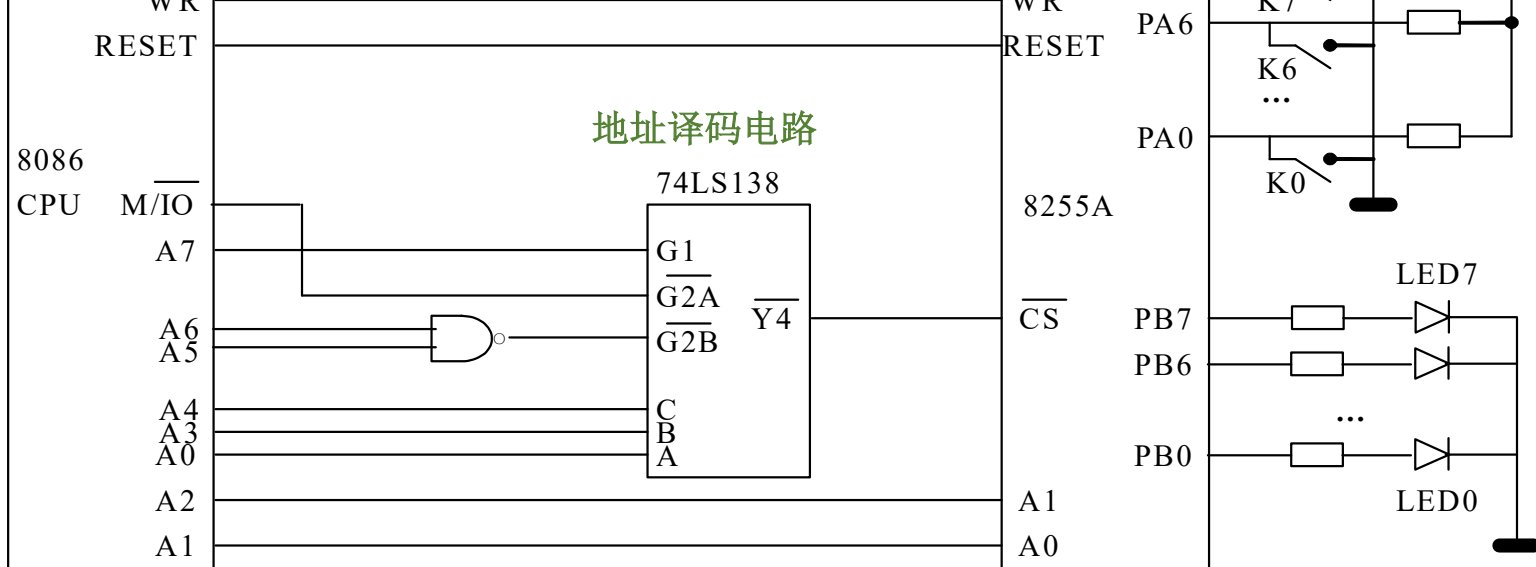
- 基本输入输出
- 数码管显示
- 键盘应用



6.3.1 基本输入输出

- 编程实现以下功能：循环检测开关状态，然后用LED显示开关状态（开关闭合则相应的LED亮）



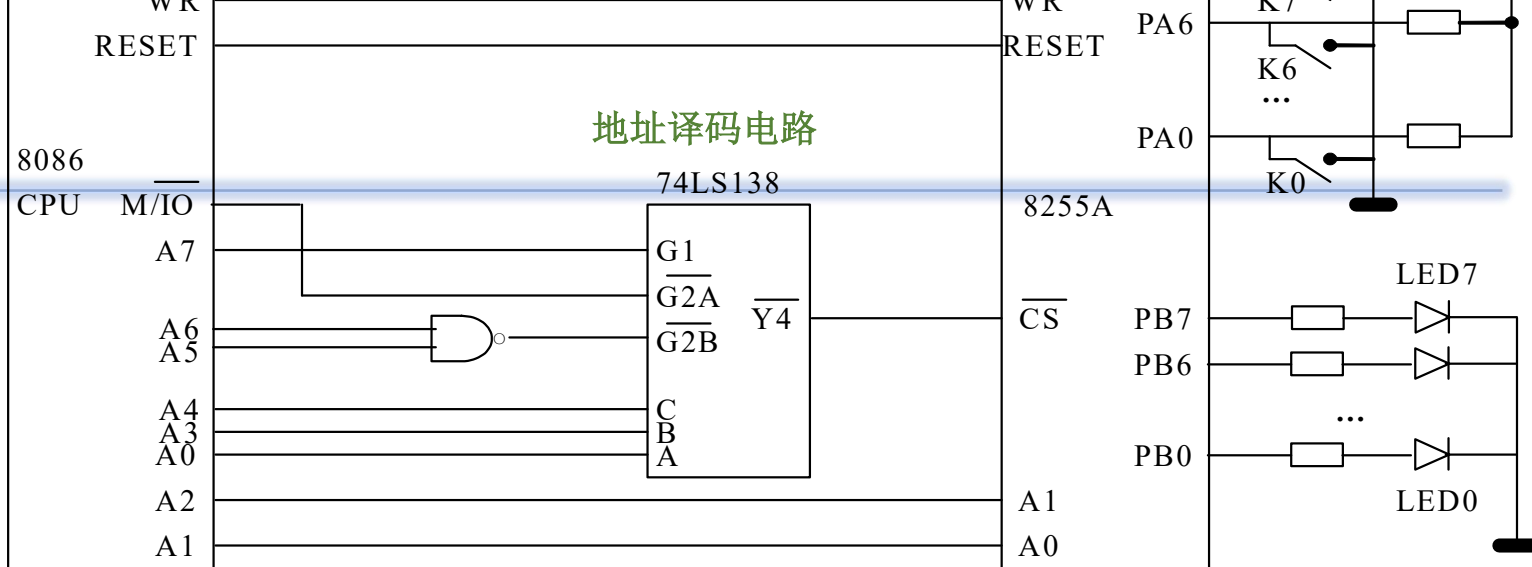


分析电路

- ① 确定端口地址。为使 $\overline{Y4} = 0$ ，74LS138的引脚应当满足以下条件：
 （A2、A1和片选信号 $\overline{Y4}$ 无关，写上去只是为了便于确定端口地址）

	0	1	0	1	0		0	
	$\overline{G2A}$	G1	$\overline{G2B}$	C	B		A	
	$\overline{M}/\overline{IO}$	A7	A6	A5	A4	A3	A2	A1
IN/OUT(0)	1	1	1	1	0	0	0	0
		同上				0	1	同上
		同上				1	0	同上
		同上				1	1	同上

$\overline{Y4} = 0$
F0H--PA端口
F2H--PB端口
F4H--PC端口
F6H--控制端口



分析电路

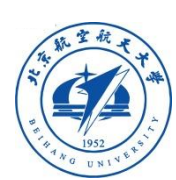
- ① 确定端口地址。为使 $\overline{Y4} = 0$ ，74LS138的引脚应当满足以下条件：
(A2、A1和片选信号 $\overline{Y4}$ 无关，写上去只是为了便于确定端口地址)

PA: F0H, PB: F2H, PC: F4H, 控制端口: F6H

- ② 确定工作方式字

标志位	A组工作方式		PA 1输入 0输出	PC _{7~4} 1输入 0输出	B组工作方式	PB 1输入 0输出	PC _{3~0} 1输入 0输出
D7=1	D6	D5	D4	D3	D2	D1	D0
1	0	0	1	1	0	0	1

99H



编程步骤

① 分析电路，确定端口

图中8255A的4个端口地址分别为F0H, F2H, F4H, F6H，对应PA、PB、PC和控制端口。

② 确定工作方式字

端口A为方式0输入，端口B为方式0输出，端口C没有使用（可设为输入），因此写入控制端口的控制字可以是10011001B（即99H）

③ 编程设置8255A工作方式

④ 编程完成指定任务

MOV AL, 99H ; 控制字

OUT 0F6H, AL ; 写入控制字

TEST_IT:

IN AL, 0F0H ; 从PA读入开关状态

NOT AL

OUT 0F2H, AL ; 端口B控制LED

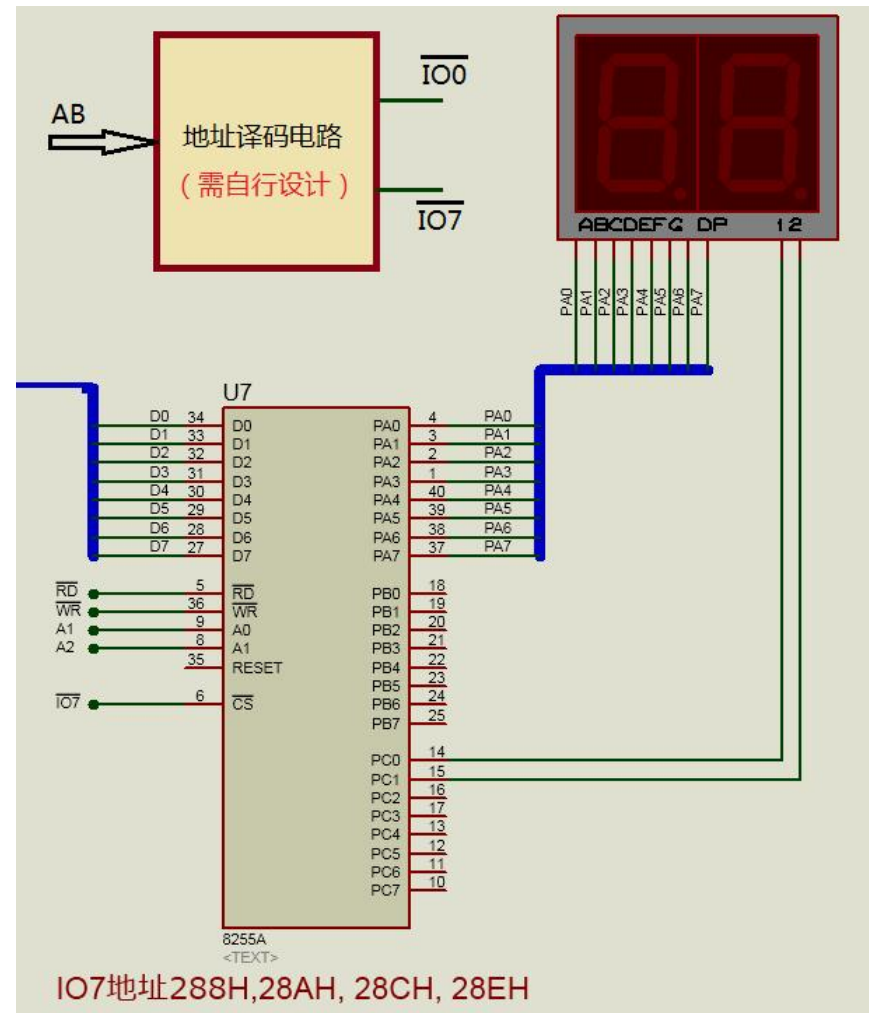
JMP TEST_IT ; 循环

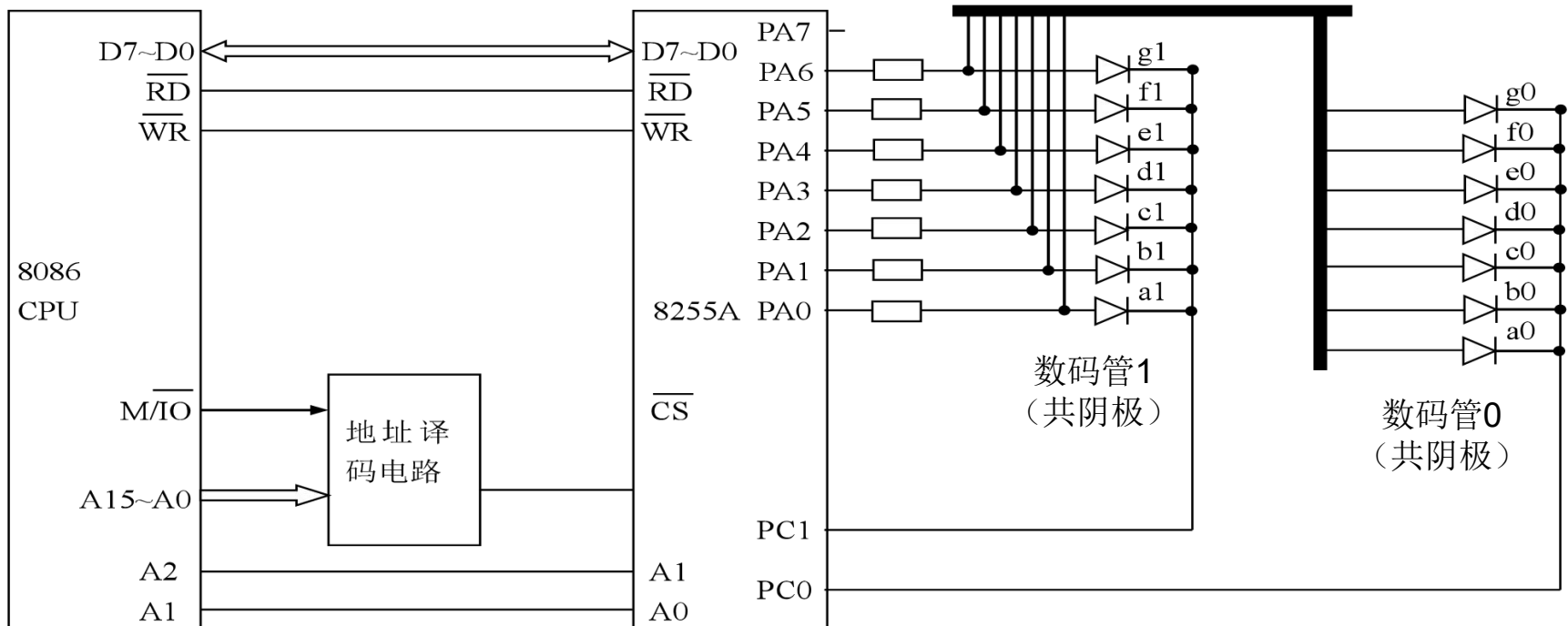


(可在最小系统EDA作业的基础上修改)

2.编程在两个数码管上分别交替显示两个数字（固定两个数字即可）。交替显示速度非常快时，两个数码管仿佛在同时显示。

3. Proteus中查找的数码管名称: 7SEG-MPX2-CC, 其字形码与实验内容1相同。

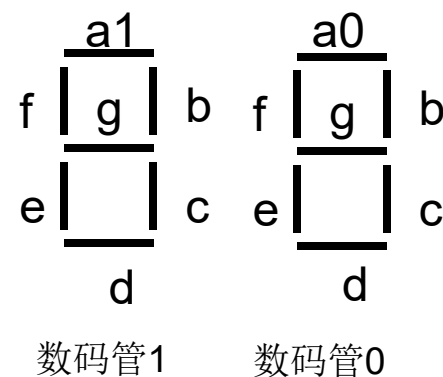




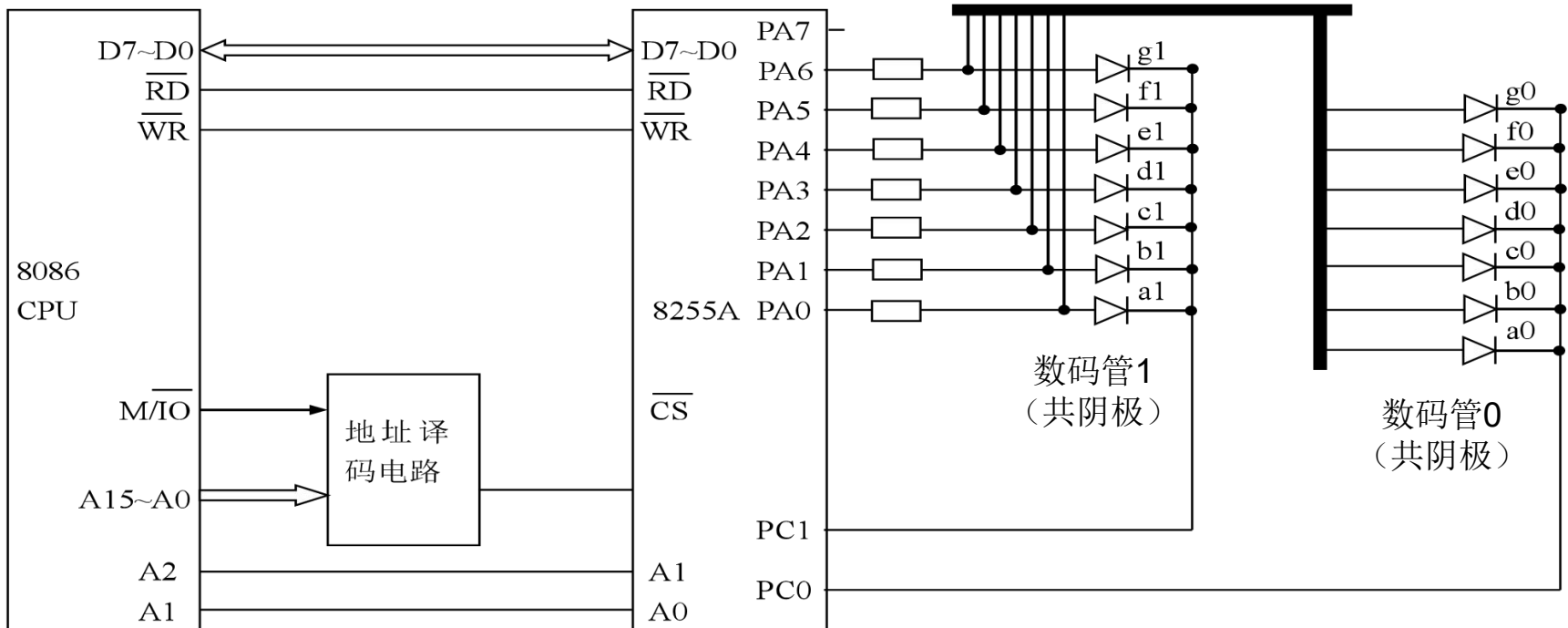
电路原理图

系统原理（交替显示两个数字）

共阴极数码管有7个LED，其阴极连接在一起。当阴极接低电平，阳极输入字形码（七段码）时，即可显示数字。



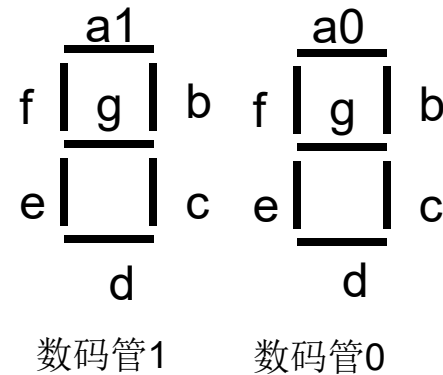
- ① 向8255A的PA端口输出X的字形码，同时令PC1 PC0=01B，则数码管1#显示X，0#灭；
- ② 向8255A的PA端口输出Y的字形码，同时令PC1 PC0=10B，则数码管1#灭，0#显示Y；
- ③ 返回①，循环。即可实现两个数字交替显示。



电路原理图

1. 共阴极数码管的字形码（七段码）

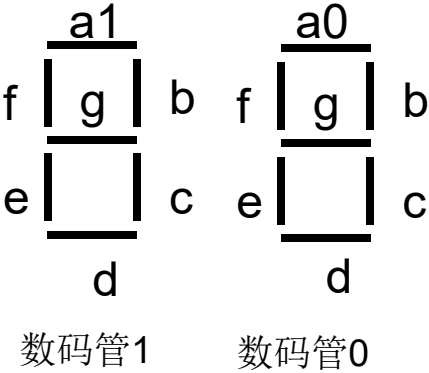
显示字形	g	f	e	d	c	b	a	七段码
0	0	1	1	1	1	1	1	3FH
1	0	0	0	0	1	1	0	06H
2	1	0	1	1	0	1	1	5BH
...								...

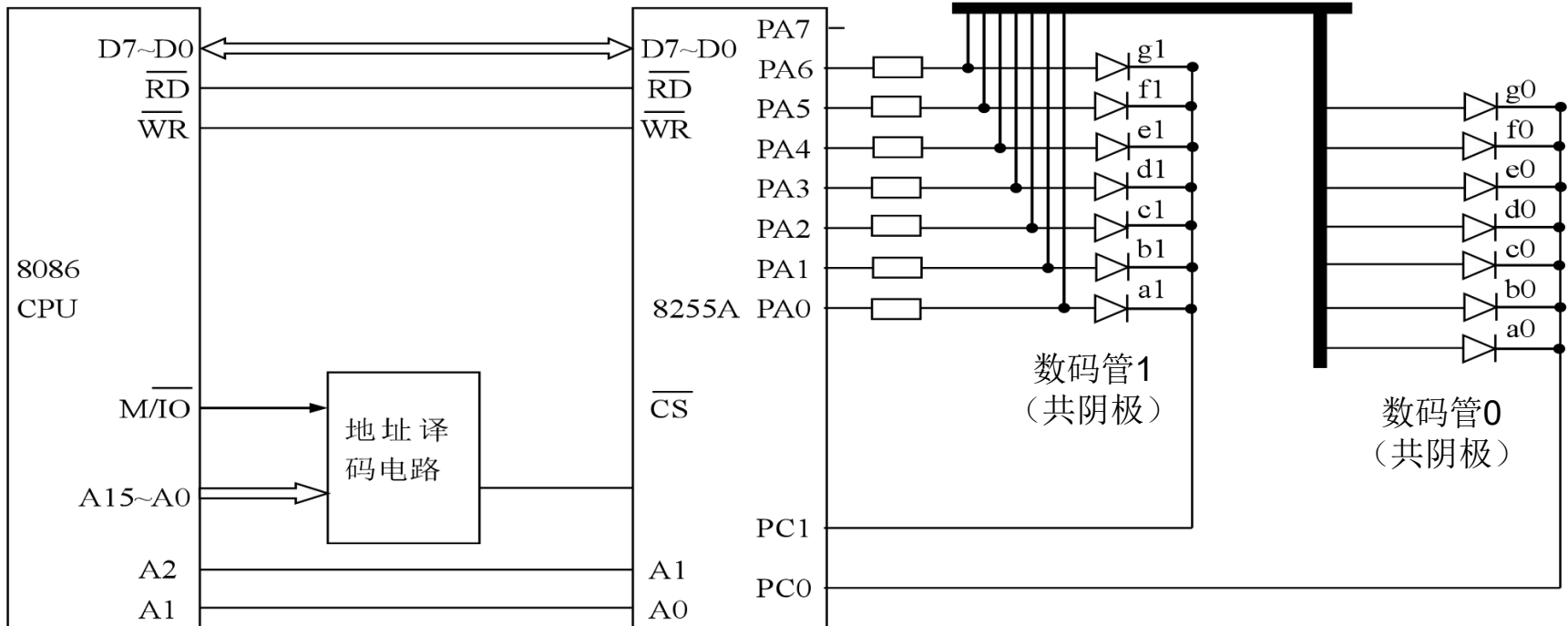


* 共阴极数码管有7个LED，其阴极连接在一起。当阴极接低电平，阳极输入字形码时，即可显示数字。

共阴极数码管字形码（七段码）表

显示字形	g	f	e	d	c	b	a	段码
0	0	1	1	1	1	1	1	3FH
1	0	0	0	0	1	1	0	06H
2	1	0	1	1	0	1	1	5BH
3	1	0	0	1	1	1	1	4FH
4	1	1	0	0	1	1	0	66H
5	1	1	0	1	1	0	1	6DH
6	1	1	1	1	1	0	1	7DH
7	0	0	0	0	1	1	1	07H
8	1	1	1	1	1	1	1	7FH
9	1	1	0	1	1	1	1	6FH
A	1	1	1	0	1	1	1	77H
B	1	1	1	1	1	0	0	7CH
C	0	1	1	1	0	0	1	39H
D	1	0	1	1	1	1	0	5EH
E	1	1	1	1	0	0	1	79H
F	1	1	1	0	0	0	1	71H





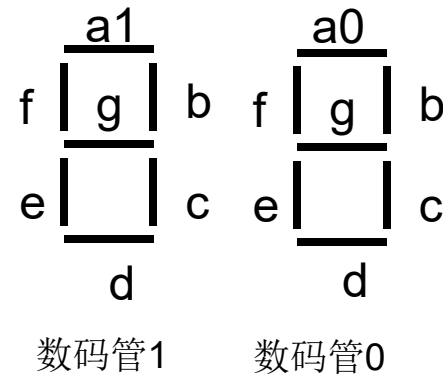
电路原理图

1. 共阴极数码管的字形码（七段码）

```

; 0 1 2 3 4 5 6 7 8 9
LIST DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH

```

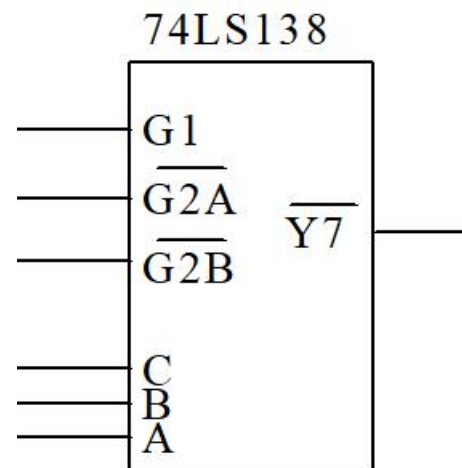
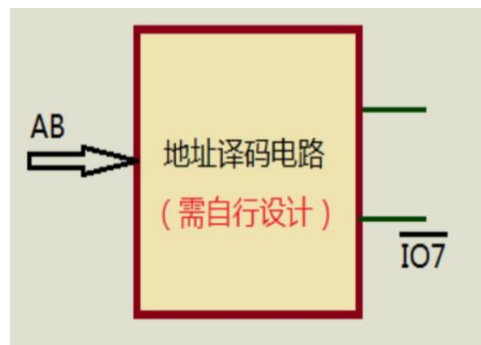


2. 地址译码器设计

使用74LS138芯片**设计**地址译码电路，使得8255A的4个端口地址分别为288H、28AH、28CH、28EH。

2. 地址译码器设计

使用74LS138芯片**设计**地址译码电路，使得8255A的4个端口地址分别为288H、28AH、28CH、28EH。



(1) 对题目要求的地址进行分析：

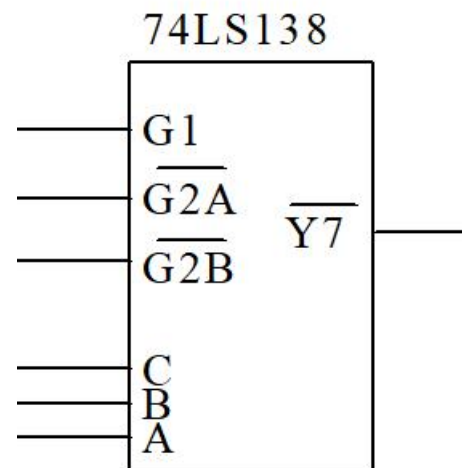
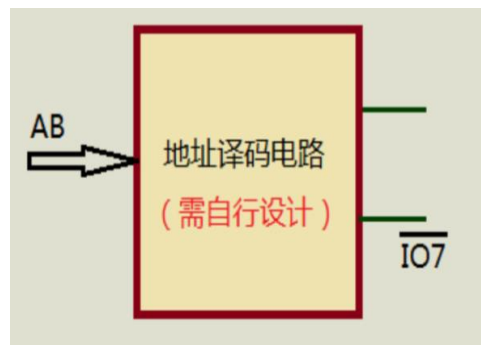
	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
288H	1	0	1	0	0	0	1	0	0	0
28AH	1	0	1	0	0	0	1	0	1	0
28CH	1	0	1	0	0	0	1	1	0	0
28EH	1	0	1	0	0	0	1	1	1	0

可发现A2,A1之外的其它地址线上的数据不变。因此可确定A2,A1连接8255A的A2,A1用于选择端口；**其它地址线（A9~A3,A0）**用于译码产生片选信号 $\overline{I07}$

* 根据电路原理图，8255A的数据引脚I连接到了地址总线（AB）低8位，因此8255A的4个端口只能使用偶地址；A2A1连接到了8255A的A1A0；和我们分析的结论一致。

2. 地址译码器设计

(2) **地址线 (A9~A3,A0)** 用于译码产生片选信号 $\overline{IO7}$, 加上 $M/\overline{IO}=0$, 即可**组合出**74LS138所需的控制信号。



已知的
信号

M/ $\overline{IO}$	A9	A8	A7	A6	A5	A4	A3			A0
0 (IN/OUT)	1	0	1	0	0	0	1			0

74LS138所
需的信号



1	0	0	1	1	1
G1	$\overline{G2A}$	$\overline{G2B}$	C	B	A



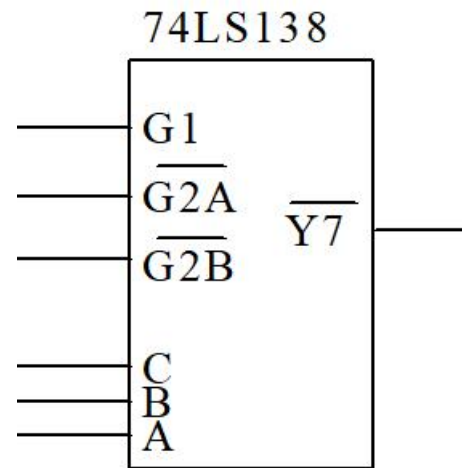
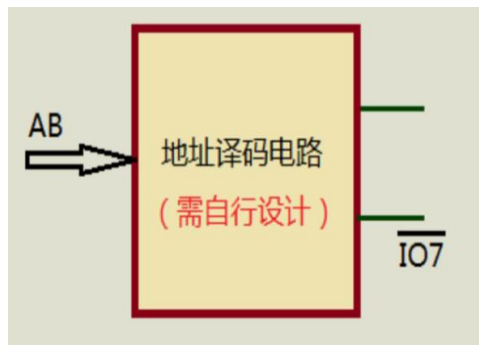
$$\overline{Y7} = \overline{IO7} = 0$$

组合连接的方式很多, 比如:

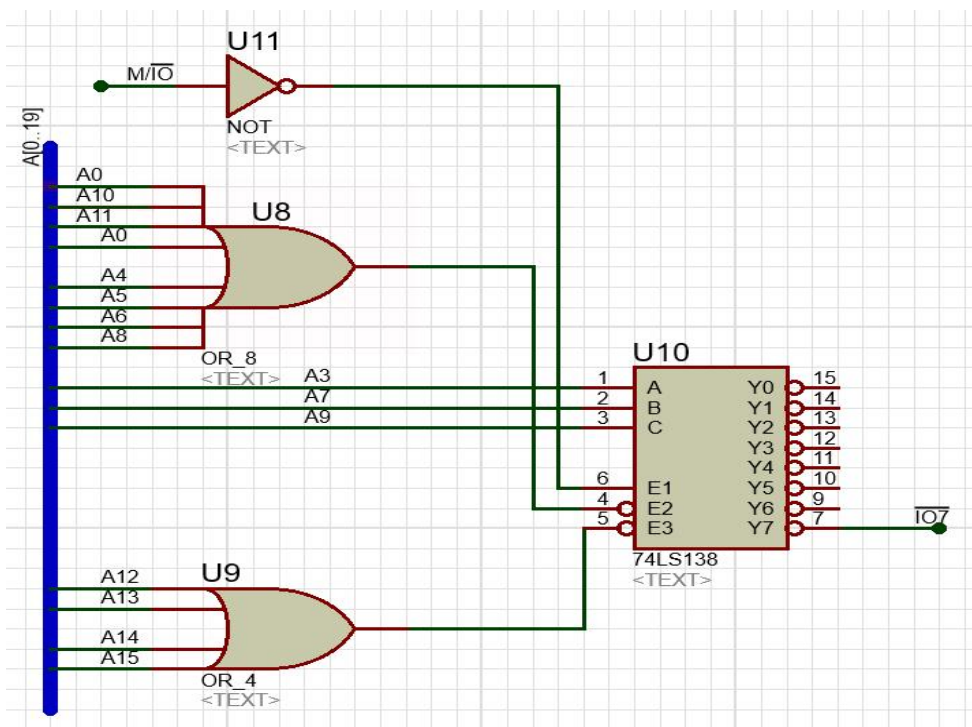
$$A9 \rightarrow G1, M/\overline{IO} \rightarrow \overline{G2A}, A8 \rightarrow \overline{G2B}, A7 \& A3 \rightarrow C, \overline{A6|A5} \rightarrow B, \overline{A4|A0} \rightarrow A$$

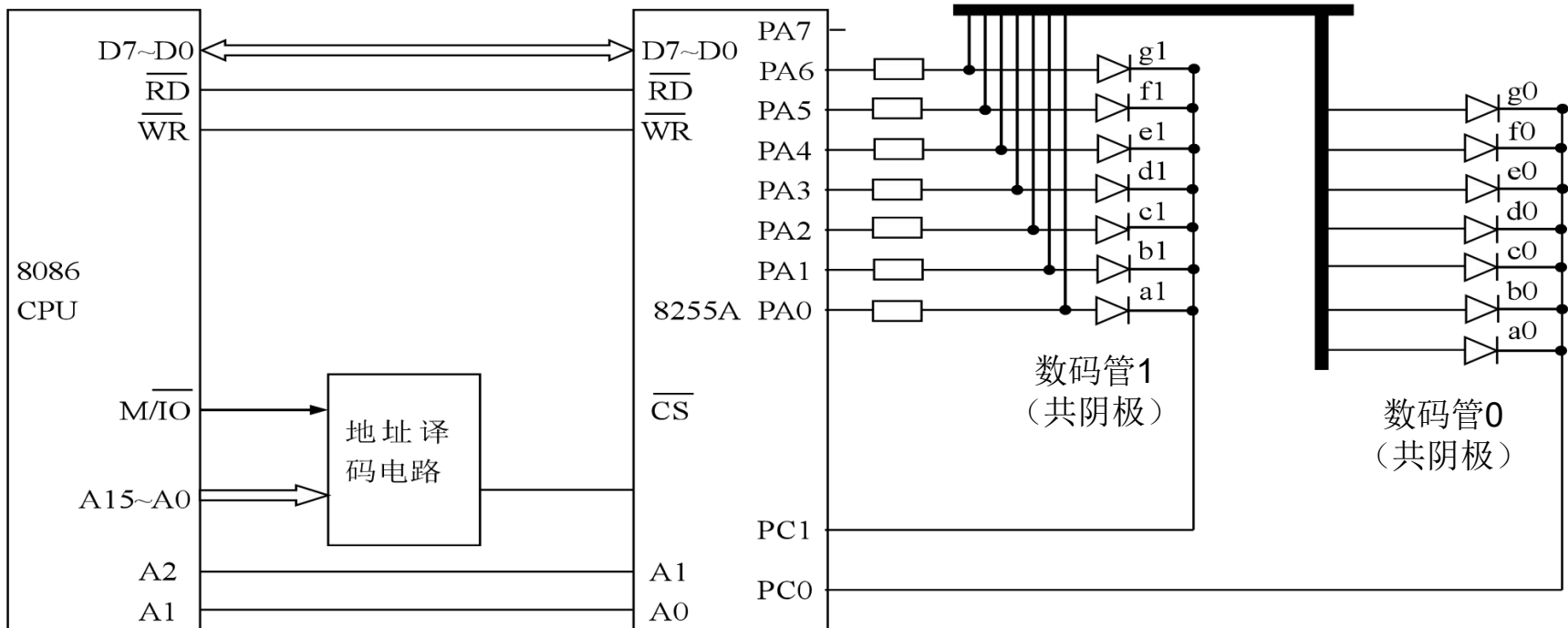
2. 地址译码器设计

(2) **地址线 (A9~A3,A0)** 用于译码产生片选信号 $\overline{IO7}$, 加上 $M/\overline{IO}=0$, 即可**组合出**74LS138所需的控制信号。



组合连接的方式很多. 也可以用下面的组合方式 (这种方式用到了所有的16条地址线, 将A15~A10上的数据按照0处理)。

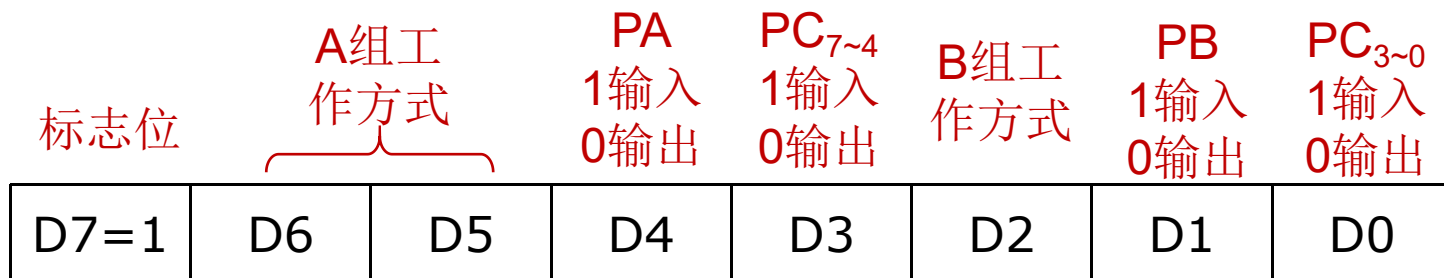




电路原理图

3. 8255A控制字

根据电路原理图和数码管驱动原理可知，8255A的PA、PC3~0应当工作在方式0，引脚为输出。PB和PC高4位未使用，可以设置为输入。



1

0

0

0

1

0

1

0

8AH

4. 程序框架（与第四章示例类似）

- 定义端口地址常量，便于编程
- 假设待显示的数字是6和3
- 注意**Protues搭建的系统**中没有DOS操作系统，所以**不能使用INT 21H**指令，程序结尾也**无需返回DOS**。

```
PORTA      EQU 288H
PORTB      EQU 28AH
PORTC      EQU 28CH
PORTCTL    EQU 28EH
```

```
DATA SEGMENT
```

```
    DT1    DB 6
```

```
    DT2    DB 3
```

```
    LIST   DB 3FH,06H,5BH,4FH,66H,
           DB 6DH,7DH,07H, 7FH,6FH
```

```
DATA ENDS
```

```
STK1      SEGMENT STACK
           ST1      DB 100 DUP(?)
STK1      ENDS
```

```
CODE      SEGMENT
ASSUME CS:CODE,DS:DATA
ASSUME SS:STK1
START:
```

```
    MOV    AX, STK1
```

```
    MOV    SS, AX
```

```
    MOV    SP, 100
```

```
    MOV    AX, DATA
```

```
    MOV    DS, AX
```

```
    ... .. ; 功能代码
```

```
    RET
```

```
CODE      ENDS
END        START
```

4.2 程序主要功能代码

(1)设置8255A工作方式

(2.1)取**DT1**，转换为七段码，输出到PA端口

(2.2)向PC端口输出01B，**1#数码管亮**，0#数码管灭

(2.3)延时，让数字持续亮一段时间

(3)向PC端口输出11B，两个数码管都灭

(4.1)取**DT2**，转换为七段码，输出到PA端口

(4.2)向PC端口输出10B，1#数码管灭，**0#数码管亮**

(4.3)延时，让数字持续亮一段时间

(5)向PC端口输出11B，两个数码管都灭

(6)返回(2.1)，循环

```
MOV    DX, PORTCTL ;(1)
MOV    AL, 8AH
OUT    DX, AL
```

```
AGN:
LEA    BX, LIST    ;(2.1)
MOV    AL, DT1
XLAT
```

```
MOV    DX, PORTA
OUT    DX, AL
```

```
MOV    DX, PORTC ;(2.2)
MOV    AL, 01B
OUT    DX, AL
```

```
CALL   DELAY      ;(2.3)
```

```
MOV    DX, PORTC ;(3)
MOV    AL, 11B
OUT    DX, AL
```

```
LEA    BX, LIST    ;(4.1)
MOV    AL, DT2
XLAT
```

```
MOV    DX, PORTA
OUT    DX, AL
```

```
MOV    DX, PORTC ;(4.2)
MOV    AL, 10B
OUT    DX, AL
```

```
CALL   DELAY      ;(4.3)
```

```
MOV    DX, PORTC ;(5)
MOV    AL, 11B
OUT    DX, AL
```

```
JMP    AGN        ;(6)
```

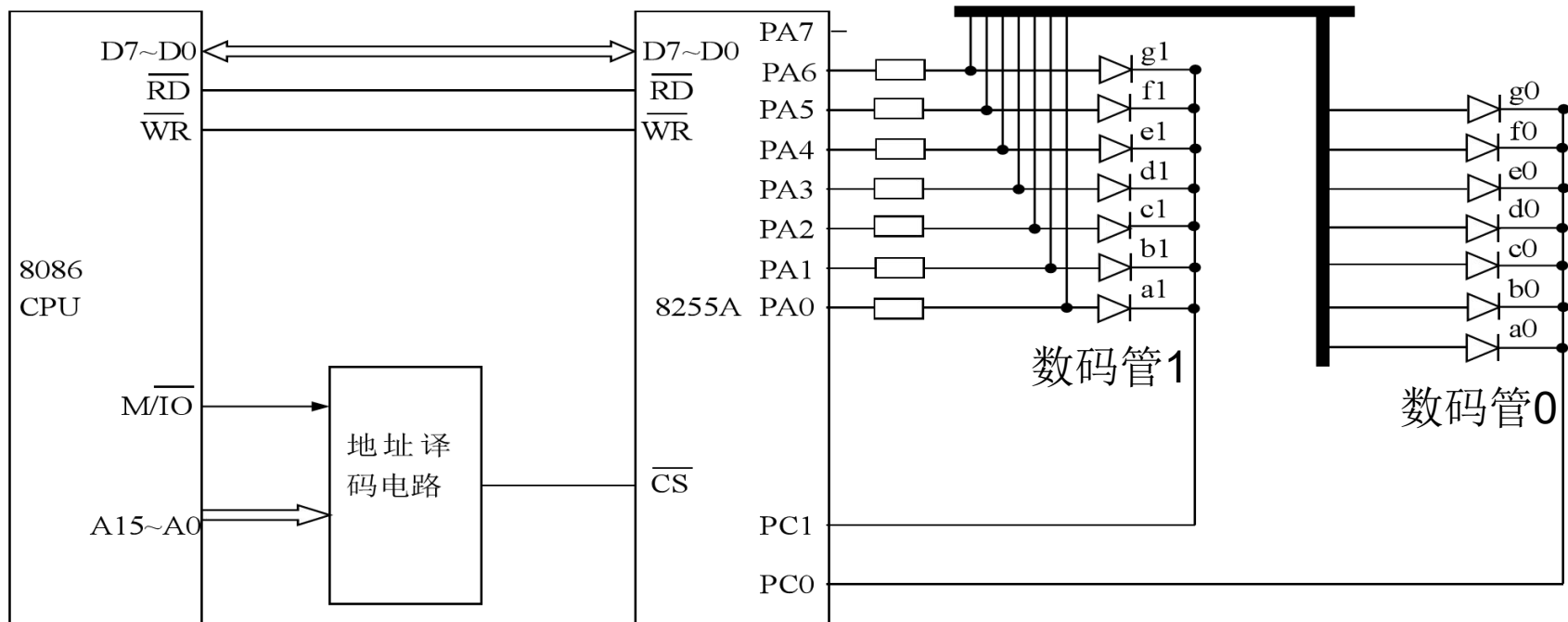
4.3 延时子程序

```
DELAY PROC
    PUSH    BX
    PUSH    CX

    MOV     BX,100
NEXT2:  MOV     CX,100
NEXT1:  LOOP    NEXT1
    DEC     BX
    JNZ     NEXT2

    POP     CX
    POP     BX
    RET
DELAY ENDP
```

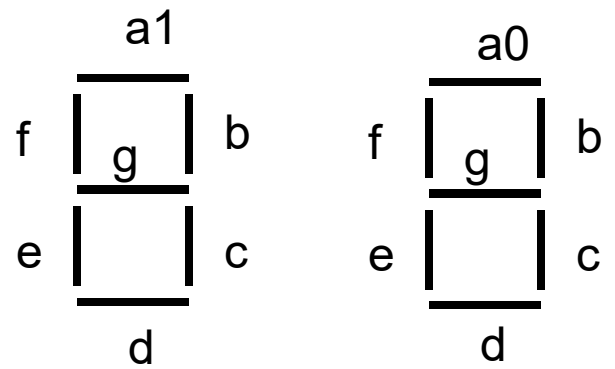
6.3.2 练习2（实验一）：数码管显示



实验内容：按照电路原理图在实验箱上连接电路，编程从键盘输入两个数字，并在数码管上显示。

8255A端口：

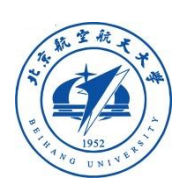
```
PORTA      EQU 288H
PORTB      EQU 289H
PORTC      EQU 28AH
PORTCTL    EQU 28BH
```



数码管1

数码管0

*由于实验箱是**模拟的8086系统**，不是真正的8086系统，所以这里的端口地址是连续的。



6.3.2 练习2（实验一）编程要求

- 定义端口地址常量

- 完整的程序框架

- 数据段和代码段定义，段分配，DS、SS初始化，返回DOS

- 步骤

除了②、④、端口地址和返回DOS，其它部分与练习1相同

① 设置8255A工作方式

② 从键盘输入两个数据，转换为对应的二进制数值

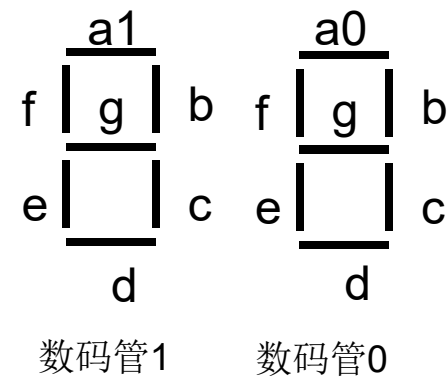
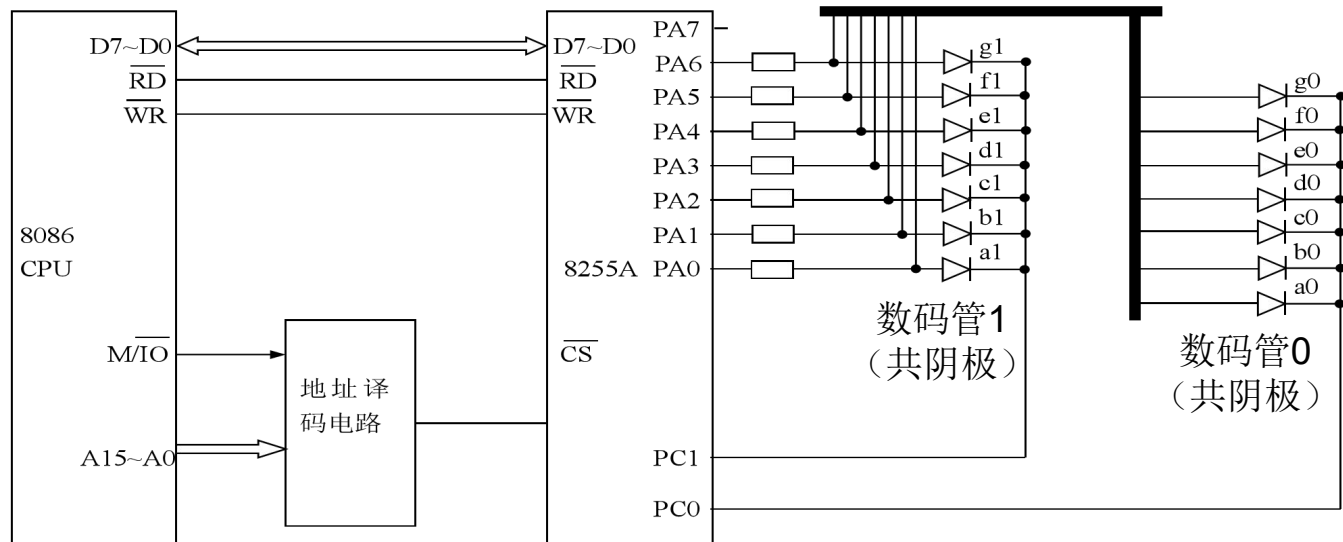
③ 将两个数字轮流输出到PORTA，

- 输出第一个数时PC1清零、PC0置1（即PC1 PC0 =01B）

- 输出第二个数时PC1置1、PC0清零（即PC1 PC0 =10B）

④ 检测键盘输入。当输入 'Q' 时退出程序，否则转至③

- 扩展：当输入为数字时，覆盖旧数字，显示新数字



■ 与练习1不同或需要添加的代码

(1) 端口地址定义

```

PORTA    EQU 288H
PORTB    EQU 289H
PORTC    EQU 28AH
PORTCTL  EQU 28BH

```

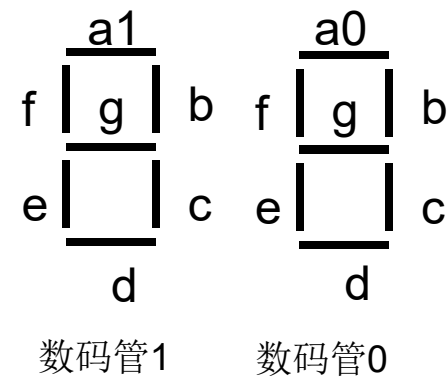
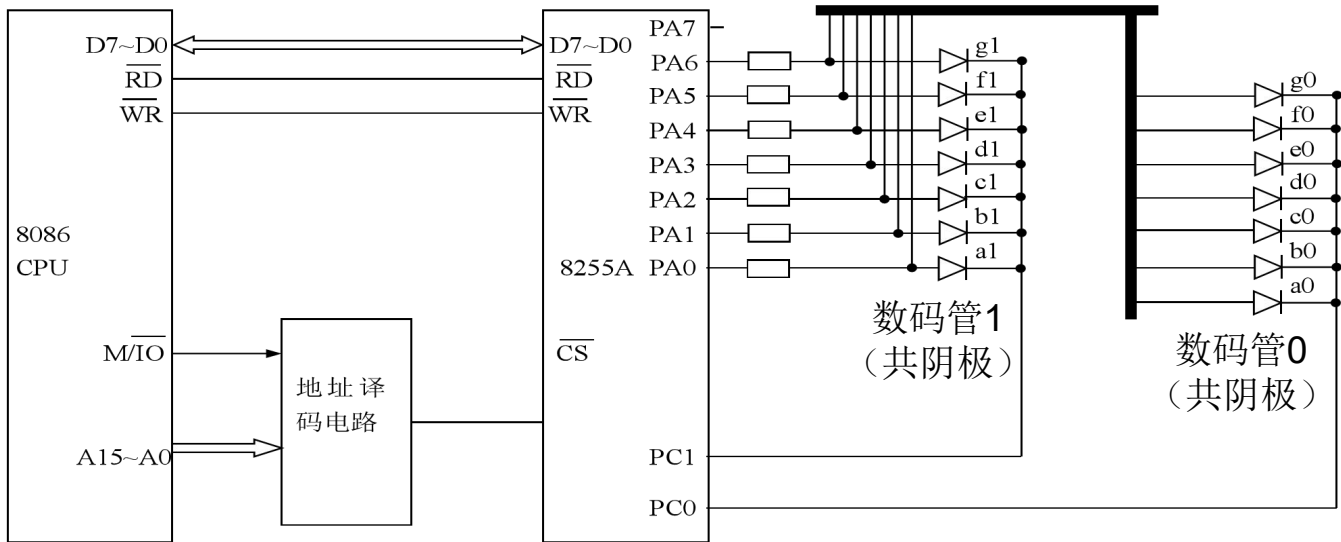
(2) 从键盘输入两个数据，转换为对应的二进制数值

```

CALL    KEYIN
AND     AL, 0FH
MOV     DT1, AL

CALL    KEYIN
AND     AL, 0FH
MOV     DT2, AL

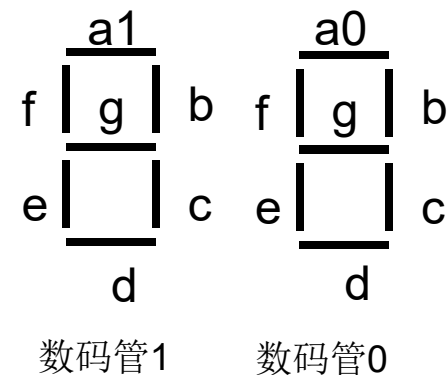
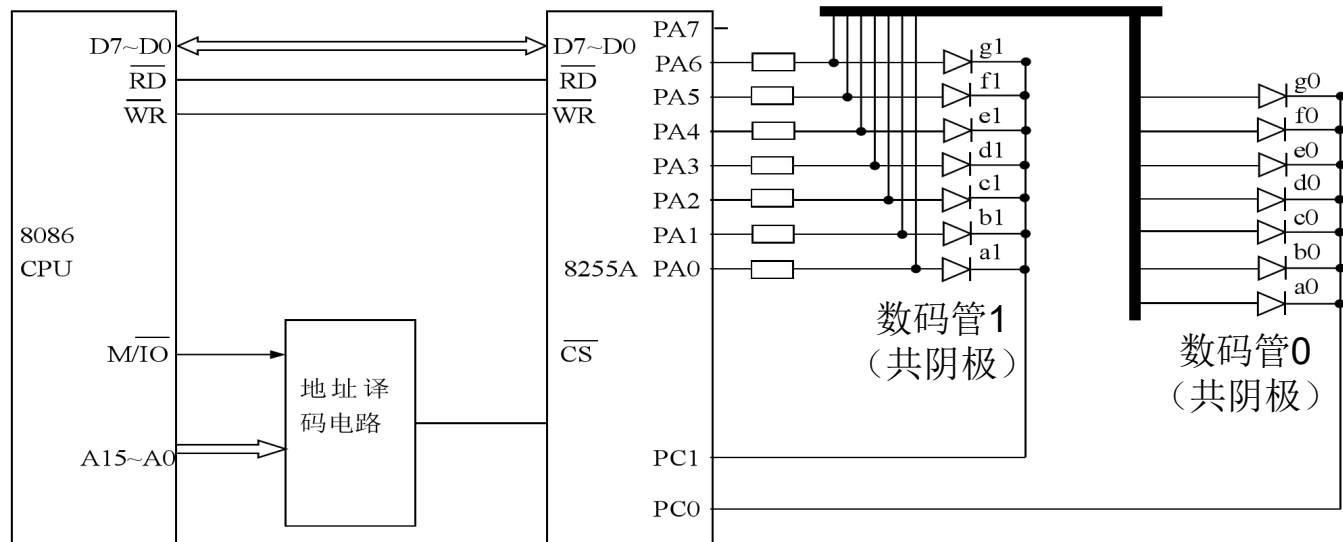
```



■ 与练习1不同或需要添加的代码

(3) 检测键盘输入。当输入 ‘Q’ 时退出程序，否则转至显示输出（即练习1代码标号AGN处）

```
MOV AH, 6
INT 21H
JZ AGN
CMP AL, 'Q'
JNE AGN
MOV AH, 4CH
INT 21H
```



■ 与练习1不同或需要添加的代码

(3) 扩展：检测键盘输入，当输入 ‘Q’ 时退出程序；当输入为数字时，覆盖旧数字，显示新数字；其它则转至显示输出（即练习1代码标号AGN处）

```
MOV AH, 6
```

```
INT 21H
```

```
JZ AGN
```

```
CMP AL, 'Q'
```

```
JE EXIT
```

```
CMP AL, 30H
```

```
JB AGN
```

```
CMP AL, 39H
```

```
JA AGN
```

```
MOV BL, DT2
```

```
MOV DT1, BL
```

```
AND AL, 0FH
```

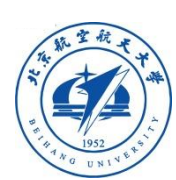
```
MOV DT2, AL
```

```
JMP AGN
```

EXIT:

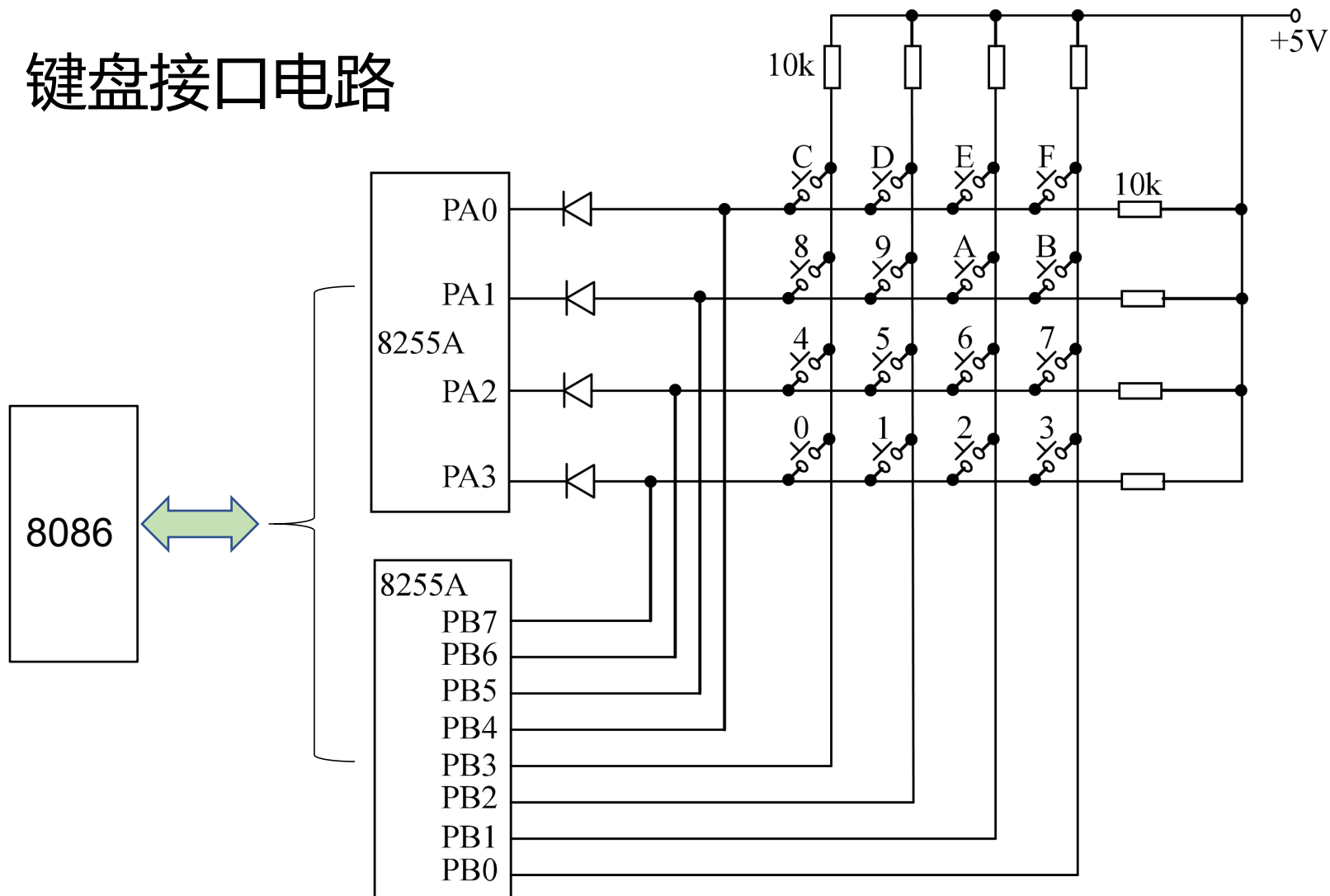
```
MOV AH, 4CH
```

```
INT 21H
```



6.3.3 键盘接口

键盘接口电路

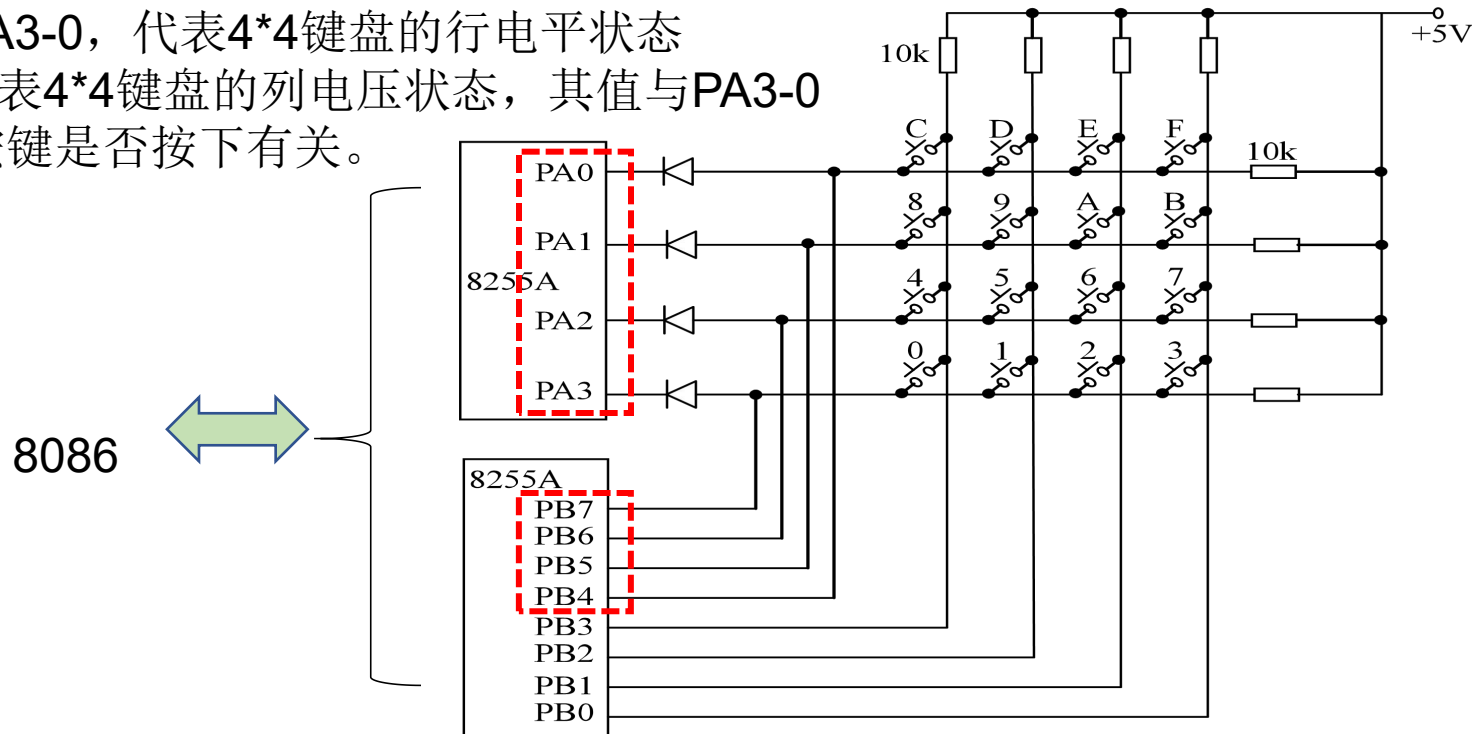


4*4键盘工作原理。根据电路图：

1) PA/PB工作在方式0；PA输出，PB输入；

2) 从PB读入的数据特点：

- PB7-4 = PA3-0，代表4*4键盘的行电平状态
- PB3-0，代表4*4键盘的列电压状态，其值与PA3-0的输出、按键是否按下有关。

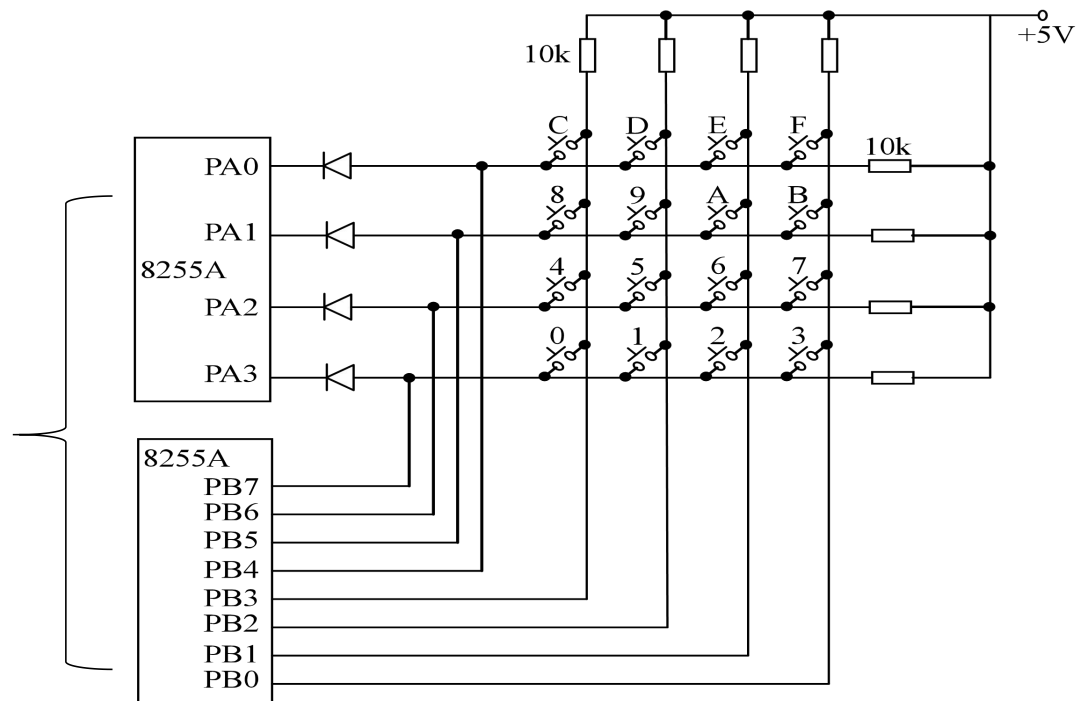


PA3-0输出1111B时，则PB3-0读入为1111B，无法据此判断是否有键按下。
判断哪个按键按下的原理：

- 1、PA3-0输出0000B，而PB3-0读入为1111B，则无键按下
- 2、PA3-0输出0000B，而PB3-0读入值不等于1111B，则有键按下（但此时无法判断哪个键按下）
- 3、确定哪个键按下：使得4*4键盘的每个行的电压依次为0（即PA3、2、1、0四个引脚依次输出0，同时其它引脚为1），读取PB。如果该行有键按下，则对应的列电平也会被拉低为0（PB3-0中对应的引脚为0）-- **扫描码**。

如何确定扫描码

8086



设端口：0FF9H~0FFFH。程序步骤：

- 1、端口定义
- 2、数据段定义
- 3、8255A初始化

4、准备工作

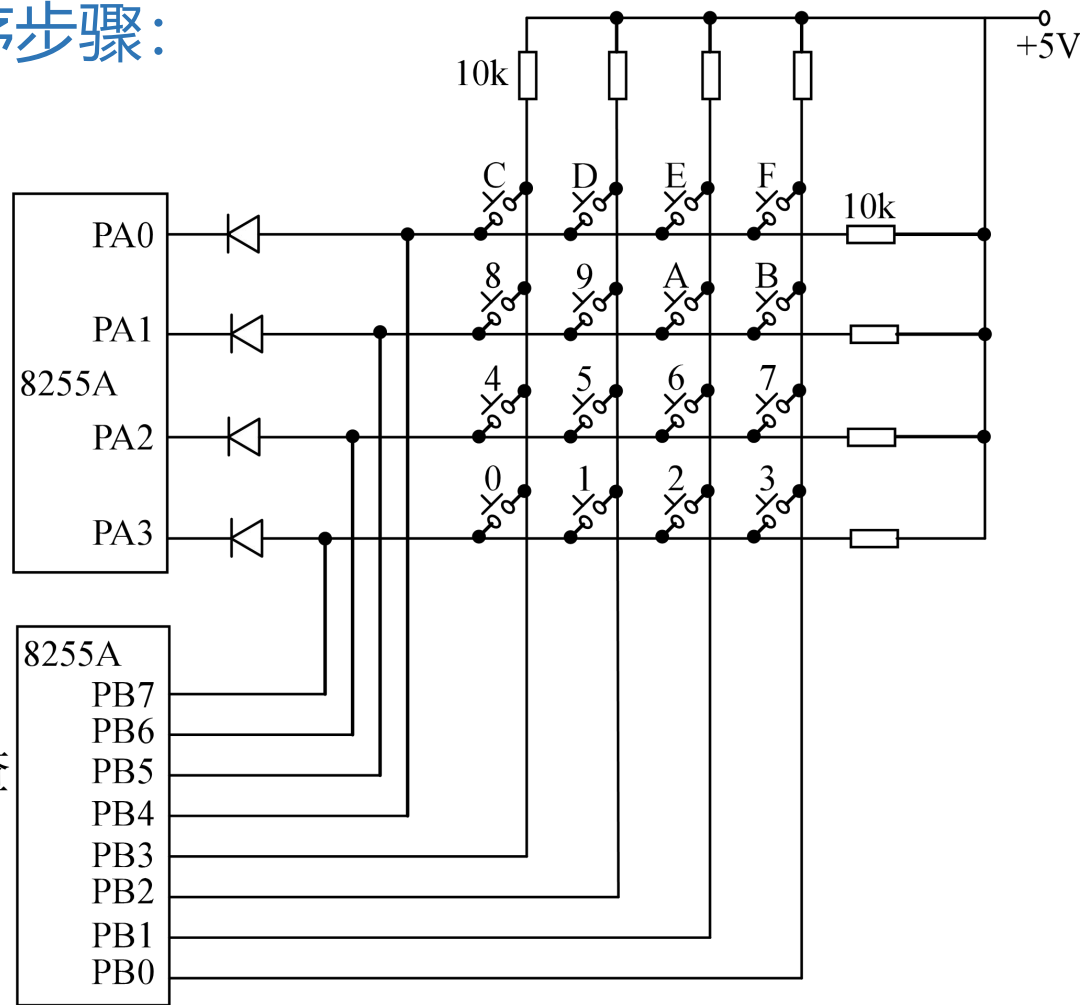
- 1) 检测是否所有键都松开，若没有则反复检测
- 2) 当所有键都松开时，再检测是否有键压下，若没有则反复检测
- 3) 若有键按下，延迟20ms再检查是否依然按下，以消除干扰。
- 如果依然有键按下，则进行步骤5

5、扫描定位

PA0~PA3依次输出0，检查PB低4位，定位按键所在的行

6、获取扫描码，并转换为键值

从PB获取扫描码，与数据段中预定义的扫描码对比，确定按键值



;端口地址

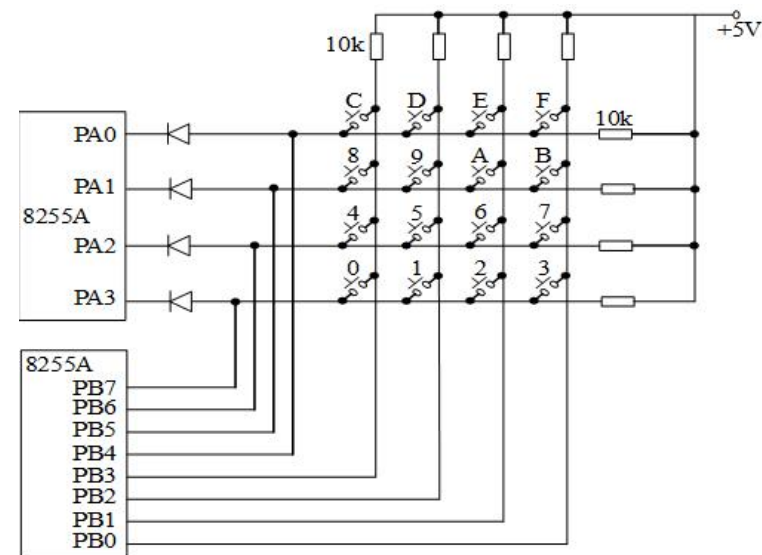
```
PORT_A      EQU    0FF9H      ;8255A端口A地址
PORT_B      EQU    0FFBH      ;8255A端口B地址
PORT_CTL    EQU    0FFFH      ;8255A控制口地址
```

;数据段， 键盘扫描码表

```
DATA        SEGMENT
;           0    1    2    3    4    5    6    7
TABLE       DB  77H,7BH,7DH,7EH,0B7H,0BBH,0BDH,0BEH
;           8    9    A    B    C    D    E    F
            DB  0D7H,0DBH,0DDH,0DEH,0E7H,0EBH,0EDH,0EEH
DATA        ENDS
```

;堆栈段

```
STK1        SEGMENT STACK
SDT         DW  50 DUP(0)
TOP_STAC    EQU SIZE  WORD
STK1        ENDS
```



;代码段

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STACK

```
START: MOV     AX, STACK
        MOV     SS, AX
        MOV     SP, TOP_STACK
        MOV     AX, DATA
        MOV     DS, AX
```

;初始化8255A, 方式0, A口作输出, 端口B和C作输入

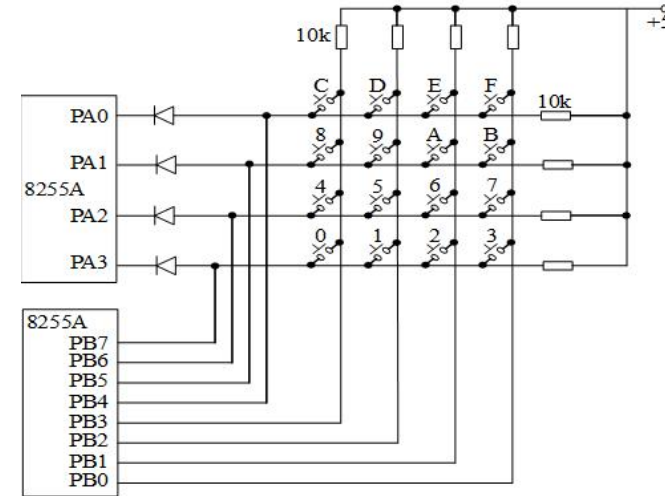
```
        MOV     DX, PORT_CTL    ;指向控制口
        MOV     AL, 1000 1011B  ;控制字
        OUT     DX, AL          ;写入控制字
```

;向所有行送0

```
        MOV     DX, PORT_A      ;
        MOV     AL, 00H
        OUT     DX, AL          ;向端口A各位输出0
```

;读列, 察看是否所有键松开

```
        MOV     DX, PORT_B
WAIT_OPEN: IN     AL, DX        ;从B口读取键盘状态
        AND     AL, 0FH         ;只查低4位
        CMP     AL, 0FH        ;是否都为1(键都松开)
        JNE     WAIT_OPEN      ;否继续查
```



;各键都松开，再查列是否有0，即：是否有键按下

```
WAIT_PRESS: IN      AL, DX                ;读B口
              AND    AL, 0FH              ;
              CMP    AL, 0FH              ;
              JE     WAIT_PRESS           ;无，等待
```

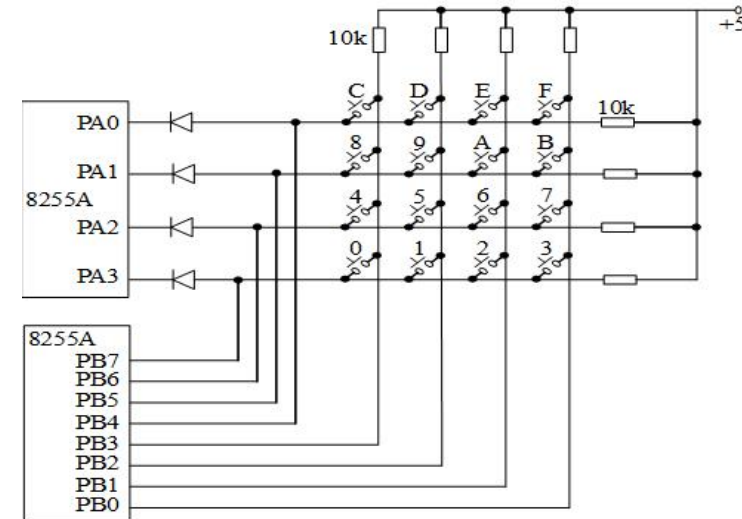
;有键按下，延时20ms，消除抖动

```
MOV    CX, 16EAH
```

```
DELAY:  LOOP DELAY    ;
```

;再查列，看键是否还按下

```
IN      AL, DX
AND     AL, 0FH
CMP     AL, 0FH
JE      WAIT_PRESS ;已松开，转去等待压键
```



;键仍然被压着，确定是哪个键按下

MOV AL, 0FEH ;先使最低位D0=0

MOV CL, AL ;CL=1111 1110B

NEXT_ROW:

MOV DX, PORT_A ;

OUT DX, AL ;向某一行输出低电平

MOV DX, PORT_B ;

IN AL, DX ;读取B口状态

AND AL, 0FH ;只截取列值

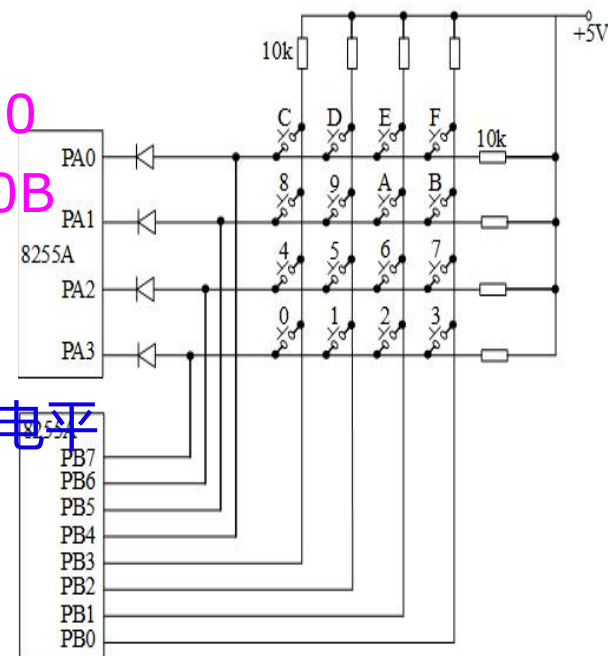
CMP AL, 0FH ;是否都是1

JNE ENCODE ;否，表示有键按下，转去编码

ROL CL, 01 ;都是1，使下行输出0

MOV AL, CL

JMP NEX_ROW ;查看下行



			0	1	2	3	4	5	6	7
;	TABLE	DB	77H,7BH,7DH,7EH,0B7H,0BBH,0BDH,0BEH							
;			8	9	A	B	C	D	E	F
		DB	0D7H,0DBH,0DDH,0DEH,0E7H,0EBH,0EDH,0EEH							

;已找到有一列为低电平，对压键的行列值编码

ENCODE:

MOV	BX, 000FH	;建立地址指针，先指向F键对应的地址
IN	AL, DX	;从B口读取行列号

NEXT_TRY:

CMP	AL, TABLE[BX];读入的行列值与表中的值比较
JE	DONE ;相等，转出
DEC	BX ;不等，指向下一个地址
JNS	NEXT_TRY ;若地址尚未减为负值，则继续
MOV	AH, 01 ;若为负值，AH置出错码01
JMP	EXIT ;退出

DONE:	MOV	AL, BL	;BL中存有键的16进制代码
	MOV	AH, 00	;AH=0,读到有效键值

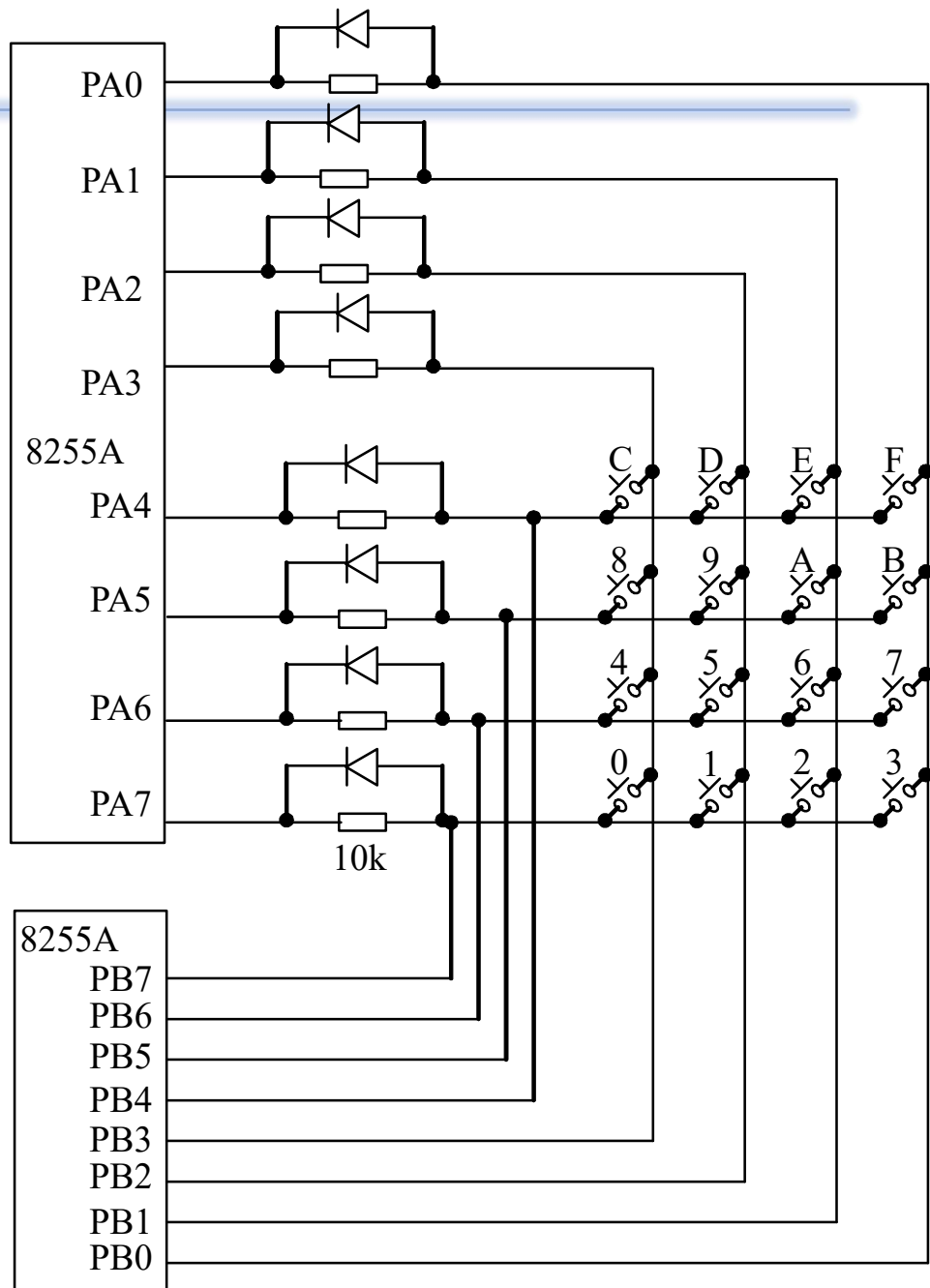
EXIT: HLT

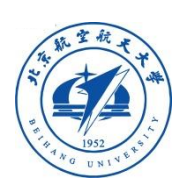
CODE ENDS
END



6.3.4 键盘接口

■ 键盘接口电路





键盘扫描

识别键盘上哪个键被按下的过程称为键盘扫描，包括以下步骤：

1. 检测是否所有键都松开了，若没有则反复检测
2. 当所有键都松开时，再检测是否有键压下，若没有则反复检测
3. 若有键按下，要消除键抖动，确认有键压下
4. 对压下的键进行编码，将该键的行列信号转换成16进制码，由此确定哪个键被压下。如果出现多键按下的情况，只有在其它键释放后，仅一个键闭合时才把此键当做本次压下的键。
5. 该键释放后，再回到2

;端口地址

PORT_A EQU 0FF9H ;8255A端口A地址

PORT_B EQU 0FFBH ;8255A端口B地址

PORT_CTL EQU 0FFFH ;8255A控制口地址

;数据段，键盘扫描码表

DATA SEGMENT

TABLE DB 77H, 7BH, 7DH, 7EH ; 0, 1, 2, 3

DB 0B7H, 0BBH, 0BDH, 0BEH ; 4, 5, 6, 7

DB 0D7H, 0DBH, 0DDH, 0DEH ; 8, 9, A, B

DB 0E7H, 0EBH, 0EDH, 0EEH ; C, D, E, F

DATA ENDS

;堆栈段

STACK SEGMENT STACK

DW 50 DUP(0)

TOP_STAC LABEL WORD

STACK ENDS

;代码段

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STACK

START: MOV AX, STACK

MOV SS, AX

MOV AX, DATA

MOV DS, AX

;初始化8255A, 方式0, A口作输出, 端口B作输入

MOV DX, PORT_CTL ;指向控制口

MOV AL, 10000010B ;控制字

OUT DX, AL ;写入控制字

; ;从行扫描开始

MOV AL, 0F0H

MOV DX, PORT_A

OUT DX, AL

;等待松开所有按键

MOV DX, PORT_B

WTRLS: IN AL, DX

CMP AL, 0F0H ;AL低4位为0

JNE WTRLS

;检查是否有按键按下

CHECKKEY: IN AL, DX

CMP AL, 0F0H

JE CHECKKEY

;等待一段时间后, 检查按键是否依然按下, 以消除抖动

MOV CX, 16EAH

DELAY: LOOP DELAY

IN AL, DX

CMP AL, 0F0H

JE CHECKKEY ;已经松开按键, 再次检查并等待

;有键按下，暂存行扫描结果

MOV BL, AL ;AL低4位为0

;扫描列

MOV AL, 0FH

MOV DX, PORT_A

OUT DX, AL

MOV DX, PORT_B

IN AL, DX ;高4位为0

;合并行扫描（BL高4位）和列扫描（AL低4位）结果

;得到按键的扫描码

OR AL, BL

;根据AL中的扫描码和TABLE，查出对应的数值

MOV BX, 0FH

NEXT_TRY: CMP AL, TABLE[BX]

JE DONE

DEC BX

JNS NEXT_TRY

MOV AH, 1

JMP EXIT

DONE: MOV AH, 0

EXIT: HLT

CODE ENDS

END START



总结

- 输入输出基本概念和功能
- 端口及IO空间，8086的IO端口地址范围
- 输入输出基本方式：程序方式（包括无条件方式和查询方式）、中断方式和DMA方式
- 8255A的引脚及内部结构
- 8255A的内部寄存器和工作方式
- 8255A应用举例